

Fachpraktikum 63585

IT-Sicherheit, IT-Forensik und Datenschutz

Tunneln von Nachrichten innerhalb von Messagebroker basierten Protokollen

Gruppenaufgabe

Per Huepenbecker

Carsten Lenzen

Robin Wetzlar



Inhaltsverzeichnis

1. Aufgabenstellung	3
2. Protokolle im Überblick	4
2.1 Corba	4
2.1.1 Funktionsweise	4
2.1.2 Typische Einsatzgebiete	5
2.1.3 Typische Paketgröße	5
2.2 MQTT	6
2.2.1 Funktionsweise	6
2.2.2 Typische Einsatzgebiete	7
2.2.3 Typische Paketgröße	8
2.3 Entscheidung	8
3 Systemdesign	9
3.1 Zielsetzung	9
3.2 Architekturübersicht	9
3.3 Kommunikationsmodell	10
3.4 Protokolldefinition und Datenformat	11
3.5 Segmentierung	13
3.5.1 Vorteile der Segmentierung	13
3.5.1 Nachteile Segmentierung	14
3.6 Einsatz eines TUN-Devices	14
3.7 Sicherheitsbetrachtung	15
4. Implementierung	15
4.1 Entwicklungsumgebung	17
4.2 Aufbau der Software	17
4.3 Installation	19
4.4 Programmablauf	21
4.5 Konfiguration und Steuerung	24
5. Tests und Analyse	25
5.1 Funktionale Tests	25
5.1.1 Aufbau des Testszenarios	25
5.1.2 Ergebnis der funktionalen Tests	26
5.2 Performance und Latenz	27
5.2.1 TCP-Durchsatz	27
5.2.2 UDP-Durchsatz, Jitter und Paketverlust	27

5.2.3 Referenzmessungen ohne Tunnel	27
5.2.4 Einordnung der Ergebnisse	27
5.3 Analyse mittels tcpdump	28
5.3.1 Analyse auf dem TUN-Interface	28
5.3.2 Analyse auf der physischen Netzwerkschnittstelle (ohne MQTT)	28
5.3.3 Analyse des MQTT-Verkehrs	28
5.3.4 Ergebnis der Analyse	28
5.4 Fehlerfälle	29
6. Fazit	30
Abbildungsverzeichnis	31
Tabellenverzeichnis	32
Quellenverzeichnis	33
Anhang	34
Anhang A: Terminalausgaben – Tunnelbetrieb	34
Performance und Latenz	41

1. Aufgabenstellung

Es gibt verschiedene Wege und Protokolle, über die sich ein Tunnel realisieren lässt. Allerdings ist es empfehlenswert, einen Tunnel über ein Protokoll zu wählen, das mit einer hohen Wahrscheinlichkeit in einem Netzwerk trotz Firewalling und vielleicht sogar irgendwelcher Proxies verfügbar ist. Das schränkt die Auswahl schon etwas ein.

Vielversprechend und entsprechend häufig genutzt sind

HTTP / HTTPS

DNS

Die wollen wir in diesem Praktikum daher nicht nutzen, das wäre ja langweilig - stattdessen wollen wir einen "Exoten" nutzen: Ein Protokoll, dass einen Message-Broker nutzt, wie

- MQTT
- CORBA

In Ihrer Gruppe wählen Sie eines dieser Protokolle oder ein weiteres broker-basiertes Protokoll aus, dokumentieren das "Warum" (es gibt kein "richtig" und "falsch", aber Ihre Beweggründe sind interessant) und entwickeln, möglichst unter Zuhilfenahme vorhandener Tools / Frameworks / bekannter Lösungen eine Implementierung, um einen Tunnel zu ermöglichen.

Im Idealfall können Sie dann durch diesen Tunnel z.B. mit SSH, OpenVPN oder Wireguard alles weitere durchtunneln, alternativ beschränken Sie sich darauf, daß Ihr Tunnel HTTP(S)-Anfragen durchleitet. Was Sie anbieten, ist eine Design-Entscheidung. Auch dafür sollten Sie die Argumente wieder dokumentieren.

Auch hier geht es um die Überlegungen, nicht um ein "richtig" oder "falsch".

Ihre Aufgaben für die Gruppenarbeit sind also:

1. Erörtern Sie, welches Protokoll Ihr Trägerprotokoll wird.
2. Erörtern Sie, ob Sie allgemein alle TCP oder UDP oder IP-Verbindungen tunneln wollen, oder nur HTTP(S).
3. Implementieren Sie Ihren Tunnel. Dokumentieren Sie Implementationsentscheidungen. Hinweis: Dazu sollten Sie auch die Anforderungen und Lösungsideen gründlich analysieren, der Anspruch ist, daß die Beschreibung zwar abstrakt, aber konkret genug ist, daß auf Ihr Darstellung aufbauend jemand die Implementierung erfolgreich starten kann. Der Teufel liegt bekanntlich im Detail, das vielleicht auch erst bei Tests und der Umsetzung auffällt, das müssen Sie nicht entdecken. Die Ergebnisse sollten sich in Ihrem Paper wiederfinden.
4. Testen Sie Ihren Tunnel gründlich. Dazu gehört auch, den Tunnel mal mit tcpdump, Wireshark o.ä. aufzuzeichnen und zu prüfen, was ein Dritter sieht.
5. Dokumentieren Sie in Ihrer Abgabe, welche Gruppenmitglieder an welchem Teil der Aufgabe mitgewirkt haben, wer für welchen Teil "den Hut auf hatte" und welchen Anteil welches Gruppenmitglied an dem Teil hatte. Im Idealfall sollten im Ergebnis alle Gruppenmitglieder etwa gleich viel geleistet haben. Die Aufgaben bearbeiten Sie in Gruppen von bis zu vier Personen. (Jedes Schild hat eine Geschichte, so auch der Hinweis, daß eine Personengruppe mindestens zwei Personen umfasst.) Das Paper (also der gedanklich-wissenschaftliche Hintergrund) geben Sie als .pdf-Datei über Moodle ab, die Implementierung im (ggf. compilierbaren) Quellcode als .tar.gz.

Sie sind frei in der Wahl der Programmiersprache(n), Sie dürfen auch auf z.B. den Code von iodine oder anderen Frameworks aufsetzen, wenn das für Sie sinnvoll erscheint. Das sind Design-Entscheidungen, die sie in Ihrem Paper diskutieren und erläutern sollten.

2. Protokolle im Überblick

Um eine Entscheidungsgrundlage zu gewährleisten werden im ersten Schritt die beiden vorgeschlagenen Protokolle mit Messagebrokercharakter in Kürze dargestellt.

Dabei werden die wesentlichen Funktionsweisen, typische Anwendungsbereiche und markante Merkmale der jeweiligen Protokolle dargestellt.

Im weiteren Verlauf wird die letztendlich getroffenen Entscheidung, welches der beiden eingangs genannten Protokolle für die weitere Bearbeitung der Aufgabenstellung Relevanz findet. Diese Entscheidung wird anhand der aufgezeigten Charakteristika begründet.

2.1 Corba

CORBA (Common Object Request Broker Architecture) ist kein einzelnes Protokoll im herkömmlichen Sinne, sondern kann als Standard für eine Middlewarearchitektur, welche eine Kommunikation zwischen verteilten Softwarekomponenten ermöglicht, verstanden werden. Diese ist dabei unabhängig von der jeweilig gewählten Programmiersprache, Betriebssystem oder Hardwareplattform. Das spezifische Netzwerkprotokoll, das CORBA für die Interoperabilität verwendet, heißt Internet Inter-ORB Protocol (IIOP) und basiert auf TCP/IP. ist eine objektorientierte Middleware für verteilte heterogene Systeme, die von der Object Management Group entworfen und spezifiziert wurde [1,2,3]

2.1.1 Funktionsweise

Das Kernstück der CORBA-Architektur ist der Object Request Broker (ORB). Der ORB ist verantwortlich für die Vermittlung aller Anfragen und Antworten zwischen Clients (Nutzern eines Dienstes) und Servern (Anbietern eines Dienstes).

Der Prozess der Kommunikation funktioniert in mehreren Schritten:

1. Schnittstellendefinition (IDL):
Die Schnittstelle des Server-Objekts wird mithilfe der Interface Definition Language (IDL) getreu den Spezifikationen der Object Management Group festgelegt. Diese sind sprachneutral beschrieben und zeigen genau auf, welche Operationen das Objekt bereitstellt und welche Parameter benötigt werden.
2. Generierung von Stubs und Skeletons:
Aus der IDL-Definition werden sprachspezifische Codefragmente generiert:
 1. Stubs auf der Client-Seite:
Sie verbergen die Netzwerkkommunikation vor dem Client, sodass sich der Aufruf einer entfernten Methode wie der Aufruf einer lokalen Methode anfühlt.
 2. Skeletons auf der Server-Seite:
Sie nehmen die Anfragen entgegen, entpacken die Daten und rufen die tatsächliche Implementierung der Server-Logik auf.
3. Objektreferenz und Naming Service:
Der Server erstellt das Objekt und registriert es bei einem Naming Service. Der Client kann dann über diesen Dienst eine eindeutige Objektreferenz (ähnlich einer Adresse) für das benötigte Objekt erhalten.
4. Der ORB als Vermittler:
Wenn der Client eine Methode aufruft (über den Stub), fängt der lokale ORB diese Anfrage ab. Er verwendet die Objektreferenz, um das entfernte Server-Objekt im Netzwerk zu lokalisieren.
5. Datenübertragung mit IIOP:
Die Anfrage (einschließlich Methodennamen und Parameter) wird vom Client-ORB serialisiert (gemarshallt)¹ und über das Netzwerk mithilfe des IIOP an den Server-ORB gesendet. IIOP verwendet hierfür TCP/IP.

¹ Marshalling ist der Prozess der Transformation von Typen, wenn sie zwischen verwaltetem und systemeigenem Code wechseln müssen.[4]

6. Ausführung und Antwort:

Der Server-ORB empfängt die Daten, leitet sie an das Skeleton weiter, das die tatsächliche Methode auf dem Server-Objekt ausführt. Die Antwort wird dann auf dem gleichen Weg zurück an den Client gesendet.

Zusammenfassend ermöglicht CORBA die Transparenz der Lokalität, sodass Entwickler verteilte Anwendungen schreiben können und dabei die Annahme treffen können, dass sich alle Objekte verhalten, als ob sie sich im selben Speicherraum befinden. [1,2,3]

2.1.2 Typische Einsatzgebiete

Das CORBA-Protokoll wird heutzutage vor allem in spezifischen Bereichen und für die Integration von älteren Systemen (Altsystemen) genutzt. Obwohl es in neuen Softwareentwicklungen oft durch modernere Alternativen wie Webservices oder Microservices ersetzt wird, findet es in folgenden Bereichen weiterhin Anwendung:

- **Altsysteme (Legacy-Systeme):**

Eine der Hauptanwendungen ist die Integration bestehender, älterer Software, die auf CORBA basiert. Unternehmen, die in diese Systeme investiert haben, nutzen CORBA weiterhin, um die Funktionalität aufrechtzuerhalten und mit neuen Systemen zu verbinden.

- **Echtzeit- und eingebettete Systeme:**

System in der Luft- und Raumfahrt, wie beispielsweise die Steuerung von Flugzeugfunkgeräten oder Systeme für das Hubble-Weltraumteleskop, nutzen CORBA aufgrund seiner Zuverlässigkeit und Interoperabilität in Echtzeitumgebungen.²

- **Große Unternehmensinfrastrukturen:**

In komplexen IT-Landschaften großer Unternehmen, oft im Finanz- oder Versicherungssektor, wird CORBA weiterhin zur Kommunikation zwischen verschiedenen, heterogenen Softwarekomponenten verwendet.

- **Industrielle Automatisierung und Steuerungssysteme:**

In manchen industriellen Umgebungen wird CORBA genutzt, um verschiedene Maschinen und Steuerungssysteme plattform- und sprachunabhängig zu verbinden.

Generell lässt sich sagen, dass CORBA dort genutzt wird, wo die Migration zu neueren Technologien zu aufwändig oder riskant wäre oder wo seine spezifischen Eigenschaften, wie die garantierte Interoperabilität zwischen verschiedenen Programmiersprachen und Plattformen, nach wie vor benötigt werden. [5,6,7,8]

Da die Echtzeitfähigkeit für die Aufgabenstellung keine Relevanz darstellt, wird an dieser Stelle nicht weiter auf die Thematik und Funktionsweise des RT-Corba eingegangen.

2.1.3 Typische Paketgröße

Es gibt keine feste "typische" Größe für CORBA-Datenpakete, da diese stark von der übertragenen Datenmenge in einem Methodenaufruf abhängt. Die Paketgröße wird im Wesentlichen durch die Nutzdaten der Anwendung bestimmt, wie zum Beispiel durch die Größe von Datenstrukturen, Arrays, Sequenzen oder Dateien, die gesendet werden. CORBA verwendet das General Inter-ORB Protocol (GIOP) bzw. dessen TCP/IP-Implementierung Internet Inter-ORB Protocol (IIOP) für die Kommunikation.

Die Paketgröße der IIOP-Nachrichten steht dabei in direkter Abhängigkeit von verschiedenen Faktoren.

² Mit Hilfe einer erweiterten Implementierung namens RT-CORBA, die Systemressourcen (CPU, Netzwerk, Threads) basierend auf zugewiesenen Prioritäten vorhersagbar verwaltet und steuert.[9]

- **Anwendungsdaten:**

Dies ist der Hauptfaktor. Ein Aufruf, der nur einen einfachen Integer überträgt, erzeugt ein sehr kleines Paket. Ein Aufruf, der ein großes Array oder eine komplexe Datenstruktur mit umfangreichen Inhalten überträgt, erzeugt entsprechend größere Nachrichten.

- **Netzwerk-MTU (Maximum Transmission Unit):**

Auf der Netzwerkebene, insbesondere bei Ethernet über TCP/IP, liegt die maximale Paketgröße (MTU) üblicherweise bei etwa 1500 Bytes. Größere IIOP-Nachrichten werden automatisch vom zugrunde liegenden TCP/IP-Stack oder dem ORB selbst in mehrere kleinere IP-Pakete (Fragmentierung) zerlegt, um über das Netzwerk gesendet zu werden.

- **ORB-Konfiguration (Fragmentierung):**

Viele ORB-Implementierungen verfügen über eine Konfigurationseinstellung für die maximale Nachrichtengröße, bevor eine Fragmentierung auf GIOP-Ebene stattfindet. Der Standardwert für diese Fragmentgröße ist oft 1024 Bytes (z. B. bei IBM oder Oracle ORBs), kann aber vom Entwickler angepasst werden. Nachrichten, die größer als dieser Wert sind, werden in Fragmente aufgeteilt.

- **Header-Overhead:**

Jedes IIOP/GIOP-Paket hat einen festen Header von 12 Bytes, plus zusätzliche Overheads durch TCP (ca. 20-32 Bytes) und IP (ca. 20 Bytes).

Abschließend lässt sich sagen, dass es keine typische Größe für CORBA-Pakete. Kleine Pakete können nur wenige Dutzend Bytes groß sein, während große Datenübertragungen leicht mehrere Megabyte erreichen können und teils fragmentiert über das Netzwerk gesendet werden. Die Paketgröße steht im direkten Zusammenhang zur Datenmenge, die von der jeweiligen Anwendung über die definierte IDL-Schnittstelle gesendet werden.
[1,2,10]

2.2 MQTT

MQTT (Message Queuing Telemetry Transport) ist ein schlankes, offenes Nachrichtenprotokoll, das speziell für die Machine-to-Machine (M2M)-Kommunikation im Internet of things³ (IoT) entwickelt wurde. Es ist darauf ausgelegt, Daten effizient in Umgebungen mit begrenzter Bandbreite, hoher Latenz oder unzuverlässigen Netzwerkverbindungen zu übertragen. [11, 12]

2.2.1 Funktionsweise

Im Gegensatz zum traditionellen Client-Server-Modell (wie HTTP mit Anfrage / Antwort) basiert MQTT auf dem Publish / Subscribe-Modell (Veröffentlichen / Abonnieren). Die Kommunikation wird über eine zentrale Komponente, den sogenannten MQTT-Broker, abgewickelt.

Die Architektur umfasst drei Hauptbestandteile:

- **Publisher (Veröffentlicher):**

Ein Gerät (Client), das Daten sendet, indem es Nachrichten zu einem bestimmten Thema (Topic) an den Broker "veröffentlicht".

- **Subscriber (Abonnent):**

Ein Gerät (Client), das Nachrichten empfängt, indem es bestimmte Themen (Topics) beim Broker "abonniert".

- **Broker:**

Die zentrale Instanz, die als Vermittler fungiert. Er nimmt Nachrichten von Publishern entgegen, filtert sie nach Themen und leitet sie an alle interessierten Subscriber weiter.

³ Auch *internet der Dinge* genannt

Das System entkoppelt Sender und Empfänger räumlich und zeitlich. Das bedeutet, dass der Publisher nicht wissen muss, wer die Daten empfängt. Seitens Subscriber ist ein analoges Verhalten erkennbar, da dieser nicht wissen muss, woher die Daten stammen. [11,13]

Vereinfacht lässt sich sagen, dass in einem MQTT-Netz alle Themen (Topics), von einem Publisher gespeist und von einem Subscriber gelesen werden.

2.2.2 Typische Einsatzgebiete

MQTT wird heute als De-facto-Standard für die Datenübertragung im Internet der Dinge angesehen und findet aufgrund seiner Effizienz, Leichtgewichtigkeit und des Publish/Subscribe-Modells in zahlreichen Bereichen Anwendung. Diese erstrecken sich typischerweise auf die folgenden Anwendungsbereiche.

- **Internet der Dinge (IoT) allgemein:**
MQTT ist das führende Protokoll für die Kommunikation zwischen intelligenten Geräten, Sensoren und Aktoren. Es wurde speziell für diese Zwecke entwickelt, um Daten effizient zu übertragen.
- **Smart Home & Gebäudeautomatisierung:**
In intelligenten Häusern wird MQTT genutzt, um verschiedenste Geräte wie Beleuchtungssysteme, Thermostate, Türschlösser und Sicherheitssysteme zu vernetzen und zentral zu steuern, oft in Verbindung mit Plattformen wie Home Assistant oder OpenHAB.
- **Industrielle Automatisierung (IIoT):**
In der Fertigungsindustrie spielt MQTT eine zentrale Rolle für die Kommunikation zwischen Maschinen und Systemen. Es wird für Zustandsüberwachung (z.B. Motorstatus), vorausschauende Wartung, Energiemonitoring und die Vernetzung von Produktionsanlagen genutzt.
- **Telemetrie und Sensornetzwerke**
Ursprünglich für die Öl- und Gasindustrie zur Kommunikation über Satellit entwickelt, eignet sich MQTT perfekt für die Übertragung von Telemetriedaten (Messdaten) von entfernten oder schwer zugänglichen Sensoren.
- **Standortverfolgung und Logistik**
Systeme zur Diebstahlortung oder Standortüberwachung in der Logistik nutzen MQTT, um Positionsdaten effizient zu übermitteln.
- **Gesundheitswesen (Wearables)**
Intelligente Wearables und medizinische Sensoren, die Gesundheitsdaten sammeln und an Backend-Systeme übermitteln, verwenden häufig MQTT.
- **Cloud-Plattformen**
Große Cloud-Anbieter wie Amazon Web Services (AWS IoT), Microsoft Azure und Google Cloud bieten native Unterstützung für MQTT, um die Anbindung und Verwaltung Tausender von IoT-Geräten zu ermöglichen. [11,14,15,16]

Die Beliebtheit von MQTT in diesen Bereichen liegt in seinen Eigenschaften begründet:

- **Leichtgewichtig:**
Das Protokoll hat einen geringen Daten-Overhead und benötigt nur wenig Bandbreite, was ideal für ressourcenbeschränkte Geräte ist.
- **Effizientes Design:**
Das Publish/Subscribe-Modell entkoppelt Sender und Empfänger, was eine skalierbare und flexible Architektur ermöglicht.
- **Zuverlässigkeit:**
Es bietet verschiedene Quality-of-Service (QoS)-Stufen, um sicherzustellen, dass Nachrichten je nach Anforderung zuverlässig zugestellt werden.

2.2.3 Typische Paketgröße

Ähnlich wie bei CORBA oder HTTP gibt es keine einzelne, feste Größe für MQTT-Datenpakete. Die Paketgröße und die Menge der tatsächlichen Nutzdaten (Payload) zeigt sich anhängig von der Datenmenge, die von der jeweiligen Anwendung übertragen werden. MQTT ist jedoch darauf ausgelegt, extrem leichtgewichtig und effizient zu sein, um den Protokoll-Overhead zu minimieren.

Dabei zeigt sich der Aufbau eines MQTT-Pakets im klassischen Sinne nach dem folgenden Schema. Dabei besteht ein MQTT-Nachrichtenpaket aus den folgenden drei Teilen:

- **Fixed Header (Fester Header):**
Immer vorhanden und nur zwei Bytes groß. Er enthält grundlegende Informationen wie den Pakettyp (z.B. CONNECT, PUBLISH, SUBSCRIBE), QoS-Level und Flags.
- **Variable Header (Variabler Header):**
Seine Größe variiert je nach Pakettyp. Bei einer PUBLISH-Nachricht enthält er beispielsweise die Länge und den Namen des Topics.
- **Payload (Nutzdaten):**
Dies sind die eigentlichen Anwendungsdaten, die gesendet werden (z.B. der Sensorwert eines Temperatursensors wie {"temp": 21.5}). Dabei kann die Größe der Payload von einem einzigen Byte bis hin zu mehreren Megabyte reichen.

Die Gesamtgröße eines MQTT-Pakets wird durch die Summe dieser Teile bestimmt.

- **Mindestgröße:**
Das kleinste mögliche PUBLISH-Paket (ohne Payload, mit sehr kurzem Topic) ist nur wenige Bytes groß (oft unter 10 Bytes Overhead).
- **Typische Nutzung:**
In IoT-Anwendungen, wo oft kleine Sensorwerte oder Statusinformationen (wie ON oder OFF oder ein JSON-Objekt mit wenigen Werten) gesendet werden, sind Pakete von wenigen Dutzend Bytes bis zu einigen Kilobytes am häufigsten.
- **Maximale Größe:**
Die MQTT-Spezifikation erlaubt in der Version 3.1.1 eine maximale Paketgröße von 256 Megabyte (inklusive Header und Payload).
- **Netzwerk-Limitierung:**
Wie bei jedem TCP/IP-Protokoll werden auch große MQTT-Pakete auf der Netzwerkebene durch die Maximum Transmission Unit (MTU) – typischerweise 1500 Bytes bei Ethernet – fragmentiert und über mehrere IP-Pakete gesendet.

Zusammenfassend lässt sich sagen, dass MQTT-Pakete aufgrund ihres minimalen Overheads in der Regel sehr klein sind, die tatsächliche Größe aber in direkter Abhängigkeit der individuellen Anforderung steht.

2.3 Entscheidung

Auf Grund der dargestellten Informationslage haben wir uns innerhalb der Gruppenarbeit auf die Nutzung des MQTT Protokolls geeinigt.

Die Begründung liegt dabei hauptsächlich in dem breiteren Anwendungsspektrum gepaart mit dem gleichzeitig effizient gestalteten Nachrichtenaufbau.

Zeitgleich dient die prominente Verbreitung des MQTT-Protokolls innerhalb der IoT-Thematik vermehrte Berührungspunkte im alltäglichen Leben

Der stetig steigende Trend beim Einsatz sogenannter Smarthome-Komponenten in Verbindung mit den verschiedensten Steuerungszentralen, welche zum Beispiel auf OpenSource-Plattformen wie Home-assistant⁴ oder IO-Broker⁵ basieren, birgt weiteres Potential zur gesteigerten Verbreitung des MQTT-Protokolls und damit größeren Einfluss im privaten Bereich.

⁴ <https://www.home-assistant.io/>

⁵ <https://www.iobroker.net/?lang=de>

3 Systemdesign

Um die prinzipielle Funktionsweise, welche in Kapitel 1 laut Aufgabenstellung gefordert wird, zu erhalten, wird im ersten Schritt ein Aufbau nach Minimalkonfiguration angestrebt. Diese wird im weiteren Verlauf genauer erläutert.

3.1 Zielsetzung

Ziel dieses Projekts ist die Entwicklung und Analyse eines Netzwerktunnels, welcher TCP-basierte Kommunikation über ein Message-Broker-basiertes Protokoll transportiert. Als Trägerprotokoll wird MQTT eingesetzt.

Der Tunnel soll es ermöglichen, TCP-Verbindungen transparent über MQTT zu kapseln, sodass für einen externen Beobachter ausschließlich MQTT-Kommunikation zwischen Client und Broker sichtbar ist. Die eigentlichen TCP-Verbindungen sollen dabei auf Netzwerkebene verborgen bleiben.

Der Schwerpunkt der Arbeit liegt auf der technischen Umsetzung des Tunnels, den zugrunde liegenden Architekturentscheidungen sowie der Analyse der Sichtbarkeit des resultierenden Datenverkehrs mittels Netzwerkanalysewerkzeugen wie tcpdump oder Wireshark.

Eine stark vereinfachte Darstellung des angestrebten Informationsaustauschs ist in Abbildung 1 zu sehen. Dort werden die MQTT-Clients über eine entsprechende Datenverbindung mit dem MQTT-Broker verknüpft. Dieser kontrolliert den Datenfluss anhand der vorhandenen Topics und den zugehörigen Subscribern und Publishern.

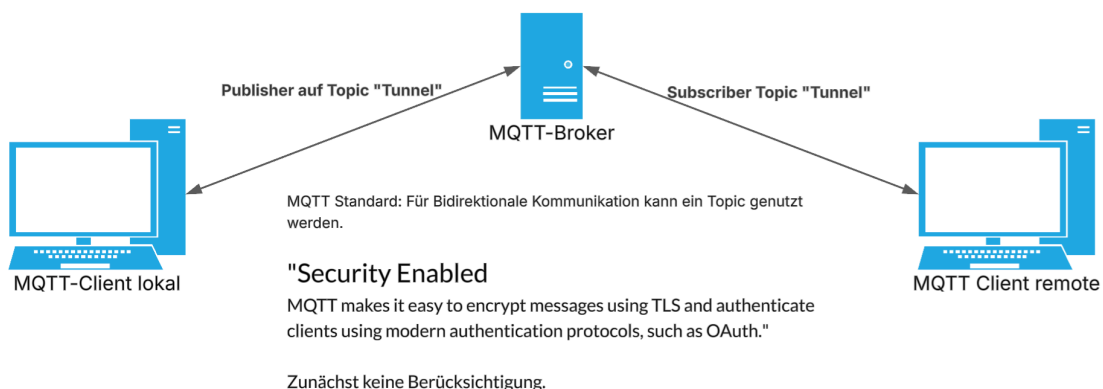


Abbildung 1 Übersicht MQTT-Clients und MQTT-Broker

3.2 Architekturübersicht

Bedingt durch die Entwicklung einer eigenen Anwendung zum Tunneln von Daten via MQTT bietet sich der Einsatz einer TUN-Device an. Ein TUN-Gerät ist eine spezifische Art von virtuellem Netzwerkgerät im Linux-Kernel, das zur Implementierung von Netzwerk-Tunneln verwendet wird, insbesondere für VPN-Dienste (Virtual Private Network). Es arbeitet auf der Netzwerkschicht (Schicht 3) und verarbeitet IP-Pakete. [19]

Über diese TUN-Devices lässt sich ein virtueller Netzwerktunnel zwischen einzelnen Teilnehmern aufbauen. Dabei läuft die Kommunikation innerhalb des Tunnels unabhängig von der Technologie der physischen Umgebung.

Der Kernel behandelt die Datenpakete der TUN-Device genauso, als würden diese von einem echten physischen Gerät abstammen. Sämtliche netzwerkrelevanten Funktionen wie

Routing, Fragmentierung oder die Verarbeitung von TCP- und UDP-Daten erfolgen dabei im regulären Netzwerk-Stack des Betriebssystems. Die TUN-Device selbst dient ausschließlich als virtuelle Ein- und Ausgabeschnittstelle für IP-Pakete, welche von der Tunnelanwendung verarbeitet werden. Aus diesem Grund kann an dieser Stelle auf weitere Details zu diesem Thema verzichtet werden.

Einen Entwurf zum angestrebten Funktionsablauf ist in Abbildung 2 dargestellt. Durch den Einsatz einer TUN-Device wird der TCP-Traffic der Applikation mit dem MQTT Gateway verknüpft.

Über die Publish / Subscribe Methodik wird mittels des gemeinsamen Topics ein MQTT-Netzwerk zwischen den Clients hergestellt.

Nachdem erfolgreiches Spannen des MQTT-Netzwerks beginnt der Informationsaustausch. Dazu wird der TCP-Datenstrom als MQTT-Payload verpackt und auf das entsprechende Topic veröffentlicht. Durch das Subscriben des Topics durch die Gegenstelle, empfängt dieser Client den Datenstrom. Ein Socket stellt eine Verbindung zu einer virtuellen TCP-Schnittstelle her, die wiederum als TUN-Device mit der Empfangsapplikation kommuniziert.

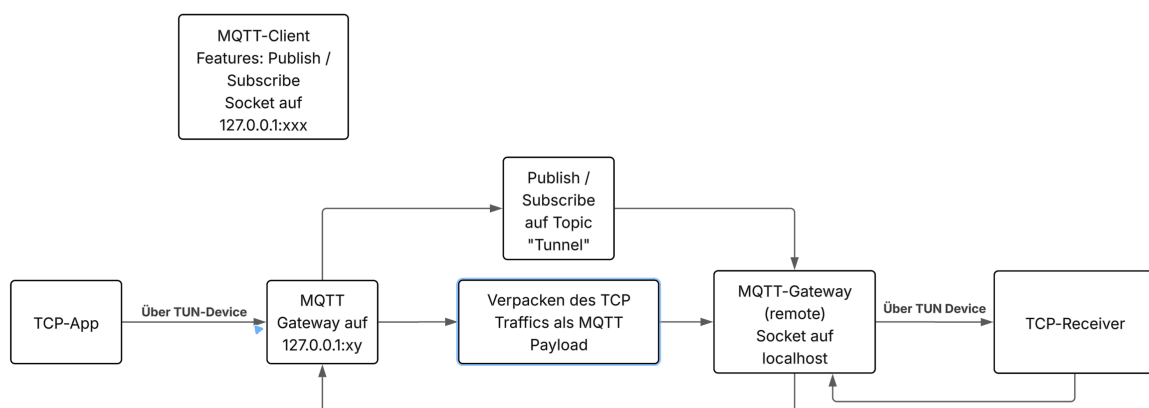


Abbildung 2 Entwurf zum initial angestrebten Funktionsablaufs

3.3 Kommunikationsmodell

Für die bidirektionale Kommunikation zwischen Tunnel-Client und Tunnel-Server werden bewusst zwei getrennte MQTT-Topics verwendet.

- **tunnel/client_to_server**
- **tunnel/server_to_client**

Dieser Informationsfluss wird in Abbildung 3 visualisiert.

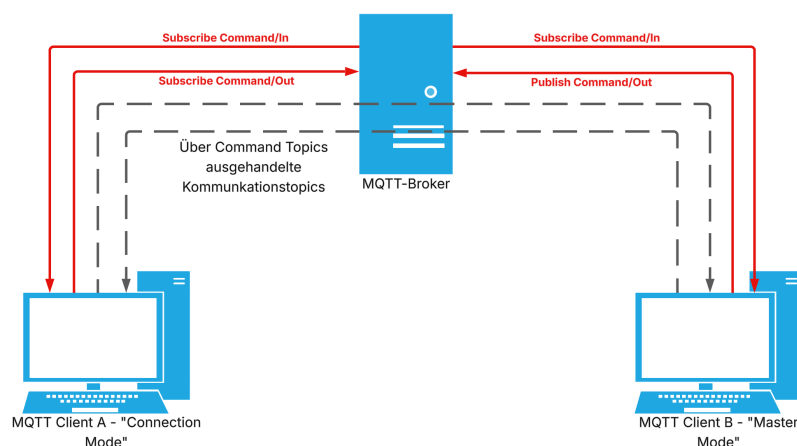


Abbildung 3 Übersicht zum Informationsfluss der MQTT-Topics

Diese Entscheidung wurde aus folgenden Gründen getroffen:

- sie verhindert zuverlässig, dass ein Client seine eigenen Nachrichten wieder empfängt
- viele verbreitete MQTT-Broker unterstützen das MQTT-5-Flag `no_local` nicht oder verarbeiten es inkonsistent
- dadurch ist die Lösung auch mit **MQTT 3.1.1** vollständig kompatibel
- das Verhalten bleibt deterministisch und unabhängig vom Broker
- die Implementierung wird robuster gegenüber unterschiedlichen Broker-Implementierungen (Mosquitto, EMQX, HiveMQ, VerneMQ usw.)

Durch diese klare Trennung der Datenrichtungen wird ein stabiler und vorhersagbarer Tunnelbetrieb gewährleistet.

Darüber hinaus steigert der Einsatz von unterschiedlichen Topics die Übersichtlichkeit im Hinblick auf eine Datenstromanalyse. Diese Datenstromanalyse erfolgt über etablierte Softwarelösungen wie TCPDump⁶ oder Wireshark⁷.

In der implementierten Testumgebung erfolgt der initiale Sitzungsaufbau über einen zentralen Command-Channel. Nach erfolgreichem Handshake werden sitzungsspezifische Topics verwendet, die anhand der Client-ID eindeutig zugeordnet sind. Für jede Sitzung existieren dabei zwei separate Topics, welche die jeweilige Kommunikationsrichtung zwischen Client und Server abbilden.

3.4 Protokolldefinition und Datenformat

Der Tunnel kapselt Netzwerkverkehr, indem er die über das TUN-Interface anfallenden IP-Pakete (Layer 3) als Nutzlast in MQTT-Nachrichten überträgt. Dadurch wird aus Sicht der Anwendungen eine normale IP-Verbindung bereitgestellt, während der Transport zwischen den Tunnelendpunkten über den MQTT-Broker erfolgt.

Die Kommunikation ist logisch in zwei Phasen aufgeteilt.

• Handshake / Sitzungsaufbau (Command-Channel)

Zu Beginn wird eine Sitzung zwischen Client und Server aufgebaut. Dabei werden Parameter wie Client-ID, die zu verwendenden Topics sowie die Tunnel-IP-Konfiguration abgestimmt. Der Server verwaltet anschließend eine Zuordnung zwischen Client-ID und Tunnel-IP und akzeptiert Datenpakete ausschließlich für die aktive Sitzung.

• Datentransfer (Data-Channel)

Nach erfolgreichem Handshake werden die rohen IP-Pakete als Payload über MQTT ausgetauscht. In der implementierten Testumgebung erfolgt dies über sitzungsspezifische Topics, z. B.:

`mqtt_tunnel/commands//A` (Richtung Server → Client)

`mqtt_tunnel/commands//B` (Richtung Client → Server)

Das Datenformat der Payload ist dabei bewusst schlank gehalten. Es werden keine zusätzlichen Protokollheader wie Sequenznummern oder Checksummen auf Anwendungsebene eingeführt. Die Integrität und Reihenfolge werden in der Praxis primär durch die darunterliegenden Protokollmechanismen (TCP-Verkehr innerhalb der IP-Pakete sowie MQTT-Transport) beeinflusst.

Da MQTT selbst auf TCP/IP aufsetzt, handelt es sich beim Tunneling technisch um eine Kapselung von TCP-Daten innerhalb der Anwendungsschicht von MQTT. [20, 21]

• Rollen:

Ein lokaler Client nimmt TCP-Verbindungen entgegen, zerlegt den Datenstrom in kleinere Pakete und sendet diese als MQTT-Nachrichten an einen MQTT-Broker. Ein entfernter Client abonniert, empfängt und setzt diese Nachrichten zu einem TCP-Datenstrom zusammen.

⁶ <https://www.tcpdump.org/>

⁷ <https://www.wireshark.org/>

- **Topic-Struktur:**

Zur Steuerung und Trennung von Verbindungen können spezifische Topics, z. B. tunnel/device_id/data für den Nutzdatentransport und tunnel/device_id/control für den Verbindungsaufbau oder -abbruch, genutzt werden.

- **Quality of Service (QoS):**

Da TCP eine verlustfreie Übertragung garantiert, wird beim Tunneling meist QoS 1 (mindestens einmal) oder QoS 2 (genau einmal) verwendet, um die Integrität des Datenstroms über den MQTT-Broker hinweg sicherzustellen.

Über den QoS wird die Zuverlässigkeit der Nachrichtenübertragung zwischen Sender (Publisher), Broker und Empfänger (Subscriber) innerhalb von MQTT definiert. Es gibt drei Stufen, die jeweils einen Kompromiss zwischen Zustellgarantie und Netzwerkbelastung darstellen. [23]

Die drei QoS-Level gliedern sich dabei wie folgt. [23]

Level	Bezeichnung	Garantie	Funktionsweise
QoS 0	At most once	Maximal einmal	"Fire and forget". Die Nachricht wird ohne Bestätigung gesendet; sie kann bei Verbindungsproblemen verloren gehen.
QoS 1	At least once	Mindestens einmal	Der Sender speichert die Nachricht, bis er ein PUBACK vom Empfänger erhält. Bei Ausbleiben wird erneut gesendet, was Duplikate erzeugen kann.
QoS 2	Exactly once	Genau einmal	Höchste Stufe durch einen vierstufigen Handshake (PUBLISH, PUBREC, PUBREL, PUBCOMP). Verhindert sowohl Verlust als auch Duplikate.

Tabelle 1 Übersicht der Quality of Service Level

- **QoS-Aushandlung:**

Der Broker verwendet für die Weiterleitung an den Subscriber das niedrigere Level aus dem Angebot des Publishers und der Anforderung des Subscribers.

- **Persistent Sessions:**

Um Nachrichten für Offline-Clients (nur bei QoS 1 & 2) zwischenspeichern, muss die Session als "persistent" konfiguriert sein.

- **Performance:**

QoS 0 ist am schnellsten und benötigt die geringste Bandbreite. QoS 2 ist aufgrund des Overheads am langsamsten und ressourcenintensivsten.

Eine Gegenüberstellung der einzelnen QoS-Levels zeigt demnach folgende Anwendungsbereiche.

- **QoS 0:**

Ideal für hochfrequente Sensordaten (z.B. Temperatur alle 100ms), bei denen der Verlust eines Einzelwerts als unkritisch bewertet wird.

- **QoS 1:**

Der Standard für die meisten IoT-Anwendungen, bei denen Daten ankommen müssen, aber die Applikation mit Duplikaten umgehen kann.

- **QoS 2:**

Erforderlich für kritische Transaktionen, wie z. B. Bestellprozesse oder Alarmer, bei denen eine Mehrfachausführung Schäden verursachen würde.

MQTT ist datenagnostisch, das heißt, es definiert kein festes Kodierungsschema für die Nutzlast (Payload). Beim Tunneling wird die Payload wie folgt strukturiert. [20, 21]

- **Payload-Inhalt:**

Die TCP-Segmente werden als binärer Datenstrom (Byte Stream) in das PUBLISH-Paket von MQTT eingebettet.

- **Header-Informationen:**

Da MQTT-Pakete keine native Sequenzierung für Anwendungsdaten besitzen, fügen Tunneling-Tools oft einen eigenen kleinen Header innerhalb der Payload hinzu, um die Reihenfolge der TCP-Datenpakete beim Empfänger sicherzustellen.

- **Payload Format Indicator:**

In modernen Implementierungen (MQTT v5.0) kann das Feld „Payload Format Indicator“ auf 0 gesetzt werden, um explizit einen nicht spezifizierten Byte-Stream zu kennzeichnen.

Der Payload Format Indicator ist eine neue Eigenschaft, die in MQTT 5.0 eingeführt wurde, um das Format der Payload in MQTT-Paketen anzuzeigen. Das Format der Nutzlast in CONNECT-, SUBSCRIBE- und UNSUBSCRIBE-Paketen ist jedoch festgelegt, so dass in der Praxis nur PUBLISH- und CONNECT-Pakete das Payload-Format deklarieren müssen.

Dieses Flag wird dem Empfänger eine Überprüfung des Nutzlastformats ermöglicht, ohne dabei den spezifischen Inhalt parsen zu müssen. Beispielsweise kann der Server überprüfen, ob die Nutzlast eine gültige UTF-8-Zeichenfolge ist. Dadurch soll eine Verteilung falsch formatierter Anwendungsnachrichten an Abonnenten vermieden werden.

Unter Abwägung der vorgangsbedingten Serverbelastung und den potentiellen Vorteilen, ist dies im Regelfall eine optionale Einstellung.

Wie bereits unter 2.2.2 geschildert, gilt der schlanke Protokollaufbau als einer der größten Vorteile von MQTT. Durch die damit verbundene Einsparung an Overhead ist ein Betrieb mit weniger leistungsschwächeren Hardware möglich.

3.5 Segmentierung

Trotz der enormen Flexibilität im Bezug auf die Nachrichtengröße innerhalb der Spezifikationen des MQTT-Protokolls empfiehlt es sich beim Tunneln von großen Datenmengen diese in kleinere Pakete aufzuteilen. [18] Diese sogenannte Segmentierung bietet mehrere Vor- aber auch Nachteile.

3.5.1 Vorteile der Segmentierung

- **Ressourcenbeschränkung**

MQTT dient oftmals als Kommunikationsprotokoll für Sensoren und Aktuatoren. Diese Geräte sind meistens mit ressourcenbeschränkten Komponenten wie Mikrocontrollern ausgestattet, die auf Grund mangelnden Zwischenspeichers mit großen Nachrichten nur bedingt umgehen können. Ein Überschreiten der hardwarebedingten Grenzen kann zu Verbindungsabbrüchen oder dem Verwerfen der Nachricht durch den Broker führen. Darüber hinaus kann eine Überlastung des Speichers dazu führen, dass die jeweiligen Komponenten zu viel Zeit für das Verarbeiten der Nachricht benötigen, wodurch das Abarbeiten der ordinär zugeordneten Hauptaufgabe beeinträchtigt oder zumindest verzögert erfolgt.

- **Netzwerkstabilität**

Große, einzelne Übertragungen sind anfälliger für Netzwerkinstabilitäten. Angenommen es tritt während der Übertragung einer großen Nachricht ein Verbindungsabbruch auf. In diesem Falle müsste die gesamte Nachricht erneut versandt werden, da MQTT keinen Mechanismus zur Fortsetzung unterbrochener Datenströme bietet. Im Falle von kleineren Segmenten, muss lediglich der fehlerhaft übertragene Datenframe erneut gesendet werden.

3.5.1 Nachteile Segmentierung

Die Nachteile der Datensegmentierung umfassen eine Reihe von Herausforderungen, die von hohen Kosten und größerem Aufwand in der Implementierung bis hin zu mangelhafter Datenqualität reichen.

- **Kosten und Ressourcenbedarf**

Der Begriff Kosten wird im weiteren mit der Belastung der Kapazität des Übertragungskanals gleichgesetzt. Erhöhte Kosten zeichnen sich für den Endanwender durch eine verminderte Übertragungsgeschwindigkeit relevanter Daten ab. Mit jedem Datensegment, welches versendet wird, müssen zusätzliche Informationen an das versandte Element angehängt werden. Diese bieten für den Nutzer keinen Mehrwert. Ein sogenannter Overhead entsteht, welcher für die erhöhten Kosten verantwortlich ist. Dieser Overhead enthält unter anderem Informationen zur originalen Nachricht, Segmentnummer oder Details zu nachfolgenden Paketen, wie z.B. letztes Segment der Ursprungsnachricht.

Wird die originale Nachricht zu fein unterteilt, steigt die Anzahl der Segmente und damit die Menge des damit verbundenen Overheads proportional. Allgemein wird dieses Phänomen als Übersegmentierung bezeichnet.

Ist die Segmentierung zu grob, treten möglicherweise die gewünschten Effekte nicht auf und es wird lediglich ein erhöhter Datenstrom verzeichnet.

- **Planung und Umsetzung**

Wie bereits erwähnt sind sowohl die zu grobe, wie auch eine zu feine Segmentierung nicht wünschenswert. Demnach kann der hinterlegte Aufteilungsprozess sehr komplex sein und erfordert sorgfältige Planung.

- **Datenqualität**

Wie bereits im Vorfeld erwähnt, bietet MQTT keinen eigenen Mechanismus zur Fortsetzung unterbrochener Datenströme. Im Falle von fehlerhaften Segmenten können demnach vereinzelte Nachrichteninhalte verloren gehen. Wird dies nicht von den entsprechenden Anwendungen erkannt oder gar korrigiert, endet die Nachrichtenübertragung in einer verminderten Datenqualität.

3.6 Einsatz eines TUN-Devices

Für die Tunnelumsetzung wird auf beiden Endpunkten ein TUN-Device verwendet. Ein TUN-Device stellt eine virtuelle Netzwerkschnittstelle auf Layer 3 bereit und liefert bzw. akzeptiert komplette IP-Pakete. Es kann vereinfacht als Punkt-zu-Punkt- oder Ethernet-Gerät betrachtet werden. Die Besonderheit liegt darin, dass anstelle einer Datenübertragung per physischen Medium, die Datenverbindung virtuell über eine eigene Anwendung aufgebaut wird. [19]

Für den Linux-Kernel wirkt diese Schnittstelle wie ein normales Netzwerkinterface. Pakete können über Routing-Regeln an das TUN-Interface gesendet werden und erscheinen dort zur Verarbeitung durch die Tunnelanwendung.

Der Ablauf gestaltet sich dabei grundsätzlich getreu dem folgendem Schema.

- **Ausgehender Verkehr**

Ein Programm (z. B. nc/SSH) erzeugt TCP-Verkehr, der vom System als IP-Pakete geroutet wird. Statt über eine physische Netzwerkkarte werden diese Pakete über das konfigurierte Routing an tun0 übergeben. Die Tunnelanwendung liest die Pakete aus tun0 und sendet sie als MQTT-Payload an den Broker.

- **Eingehender Verkehr**

Die Tunnelanwendung empfängt MQTT-Nachrichten, entnimmt die Payload (IP-Paket) und schreibt diese in tun0. Der Kernel verarbeitet die Pakete anschließend wie regulären Netzwerkverkehr, sodass die lokale Anwendung die Antworten transparent erhält.

Der Einsatz eines TUN-Devices ermöglicht es, beliebigen IP-basierten Verkehr durch den Tunnel zu leiten, ohne dass Anwendungen speziell angepasst werden müssen. Gleichzeitig wird vermieden, Ethernet-Frames (Layer 2) nachbilden zu müssen, was die Implementierung deutlich vereinfacht.

Der Kernel behandelt die Datenpakete der TUN-Device genauso, als würden diese von einem echten physischen Gerät abstammen. Die TUN-Device übernimmt dabei sämtliche Netzwerkrelevanten Aufgaben wie z.B. auch die Segmentierung des ankommenden Datenstroms. Aus diesem Grund kann an dieser Stelle auf weitere Details zu diesem Thema verzichtet werden.

3.7 Sicherheitsbetrachtung

Bei dem Protokoll MQTT liegt das Hauptaugenmerk auf eine ressourcenschonende Option zum Informationsaustausch zwischen einzelnen IoT-Geräten. Selbst auf der offiziellen Homepage von MQTT wird das Thema Sicherheit sehr vorsichtig formuliert.

You can pass a user name and password with an MQTT packet in V3.1 of the protocol. Encryption across the network can be handled with SSL, independently of the MQTT protocol itself (it is worth noting that SSL is not the lightest of protocols, and does add significant network overhead). Additional security can be added by an application encrypting data that it sends and receives, but this is not something built-in to the protocol, in order to keep it simple and lightweight.

[Originaler Auszug <https://mqtt.org/faq/> unter dem Punkt "Does MQTT support security?" 08.12.2025]

Es wird prinzipiell die Möglichkeit zur Einbindung einer SSL-Verschlüsselung ermöglicht, jedoch muss diese vom Nutzer erst aktiviert werden. Zeitgleich wird darauf hingewiesen, dass der Einsatz dieser Verschlüsselung entgegen des Hauptvorteils von MQTT wirkt und somit der Einsatz mit bedacht gewählt werden soll.

Neben der Möglichkeit zum Einsatz von Algorithmen zur Verschlüsselung der Nachrichten lassen sich zusätzlich Nutzerkennungen mitsamt entsprechender Passwörter hinterlegen. Auf diese Weise soll ein unberechtigter Lesezugriff bei versandten Nachrichten verhindert⁸ werden.

4. Implementierung

Neben der initialen Installation der Anwendung zum Tunneln von Daten via MQTT, wird in diesem Kapitel die Implementierung der Entwicklungsumgebung, ein Auszug aus dem Aufbau der Software und die finale Nutzung der Anwendung beleuchtet. Damit einher geht eine Auflistung etwaiger Abhängigkeiten, eine Erläuterung der Parameter zum Ansteuern der finalen Applikation und interne Kommunikationsmodelle zum Informationsfluss.

⁸ oder zumindest erschwert

Schritt 1: Grundlegende Build- und Systemwerkzeuge

```
sudo apt update
sudo apt install -y \
  build-essential \
  cmake \
  git \
  pkg-config \
  libssl-dev \
  libgtest-dev \
  libcereal-dev \
  libspdlog-dev
```

Schritt 2: Google Test kompilieren und installieren

```
cd /usr/src/googletest
sudo cmake -S . -B build
sudo cmake --build build -j$(nproc)
sudo cmake --install build
sudo ldconfig
```

Schritt 3: Eclipse Paho MQTT (C-Bibliothek) installieren

```
cd ~
git clone https://github.com/eclipse/paho.mqtt.c.git
cd paho.mqtt.c
cmake -S . -B build -DPAHO_WITH_SSL=ON -DPAHO_BUILD_SHARED=ON
cmake --build build -j$(nproc)
sudo cmake --install build
sudo ldconfig
```

Schritt 4: Eclipse Paho MQTT (C++-Wrapper) installieren

```
cd ~
git clone https://github.com/eclipse/paho.mqtt.cpp.git
cd paho.mqtt.cpp
cmake -S . -B build
cmake --build build -j$(nproc)
sudo cmake --install build
sudo ldconfig
```

Schritt 5: Projekt kompilieren

```
cd ~/MQTT_Tunnel
mkdir -p build
cd build
cmake ..
make -j$(nproc)
```

4.1 Entwicklungsumgebung

Die Implementierung und Erprobung des MQTT-basierten Tunnels erfolgte in einer virtualisierten Testumgebung auf Basis von Oracle VirtualBox. Hierzu wurden zwei virtuelle Maschinen mit Debian Linux eingesetzt, die identisch konfiguriert wurden.

Die virtuellen Maschinen wurden in VirtualBox unter den Namen **mqtt-vm-a** und **mqtt-vm-b** angelegt und betrieben. Beide virtuellen Maschinen verfügen über zwei Netzwerkschnittstellen, wovon eine als NAT-Interface ausgeführt ist und ermöglicht den Zugriff auf externe Netzwerke. Die zweite Schnittstelle ist als Host-Only-Netzwerk konfiguriert und dient der direkten Kommunikation zwischen den virtuellen Maschinen sowie dem Hostsystem. Die IP-Adressierung im Host-Only-Netzwerk ist statisch konfiguriert.

- **mqtt-vm-a**: 192.168.56.101
- **mqtt-vm-b**: 192.168.56.102

Der administrative Zugriff auf die Systeme erfolgt über SSH. Für die Durchführung der Tests wurden folgende Benutzerkonten verwendet:

- **mqtt-vm-a**: Benutzer usera
- **mqtt-vm-b**: Benutzer userb

Die zuvor gezeigte Anleitung beschreibt die Einrichtung der Entwicklungsumgebung unter Debian und gliedert die einzelnen Aktionen als individuell gebündelte Schritte.

4.2 Aufbau der Software

Die entwickelte Software zum Tunneln der TCP-Pakete über das MQTT-Protokoll gliedert sich in mehrere Komponenten. Diese Aufteilung gestaltet sich dabei gemäß dem in Abbildung 4 gezeigten Aufbaus.

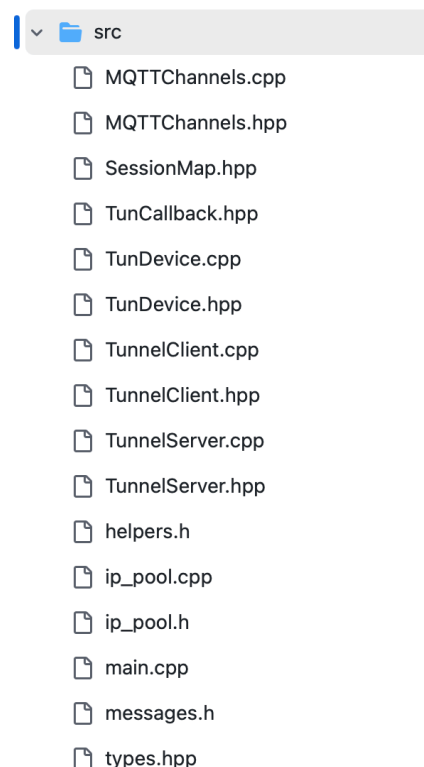


Abbildung 4 Auszug der Sourcefiles aus dem Github Repository⁹

⁹ https://github.com/PerHuepenbecker/MQTT_Tunnel/tree/main/src

In der Datei *helpers.h* ist eine Hashfunktion zur Generierung von SHA256 Schlüsseln hinterlegt. Neben der Integrität der versandten Daten, wird hiermit zeitgleich die Funktionsfähigkeit der gesamten Applikation überprüft. Der ankommende Datenstroms aus der Datenquelle muss, trotz der mehrfachen Übersetzung in die unterschiedlichen Protokolle, mit dem Ankommenden an der Datensenke übereinstimmen.

Die Dateien *ip_pool.h* und *ip_pool.cpp* halten die Klassen für das Management des potentiellen IP-Pools bereit. Beim aktuellen Softwarestand wird ein IP-Bereich /24 unterstützt, was einer Höchstgrenze von 256 Teilnehmern entspricht.

Abbildung 5 zeigt das Zusammenspiel zwischen der Klasse TunnelServer und den anderen Funktionsanteilen. Dabei implementiert die Klasse TunnelServer einen MQTT-basierten Tunnel-Server, der mehrere Tunnel-Clients verwalten kann und IP-Pakete zwischen einem lokalen TUN-Interface und MQTT-Datenkanälen weiterleitet. Sie übernimmt den vollständigen Serverbetrieb inklusive TUN-Device Setup, Session-Handshake, IP-Adressvergabe, Routing/NAT-Konfiguration bei Gatewaybetrieb, sowie optionaler Ende-zu-Ende-Verschlüsselung.

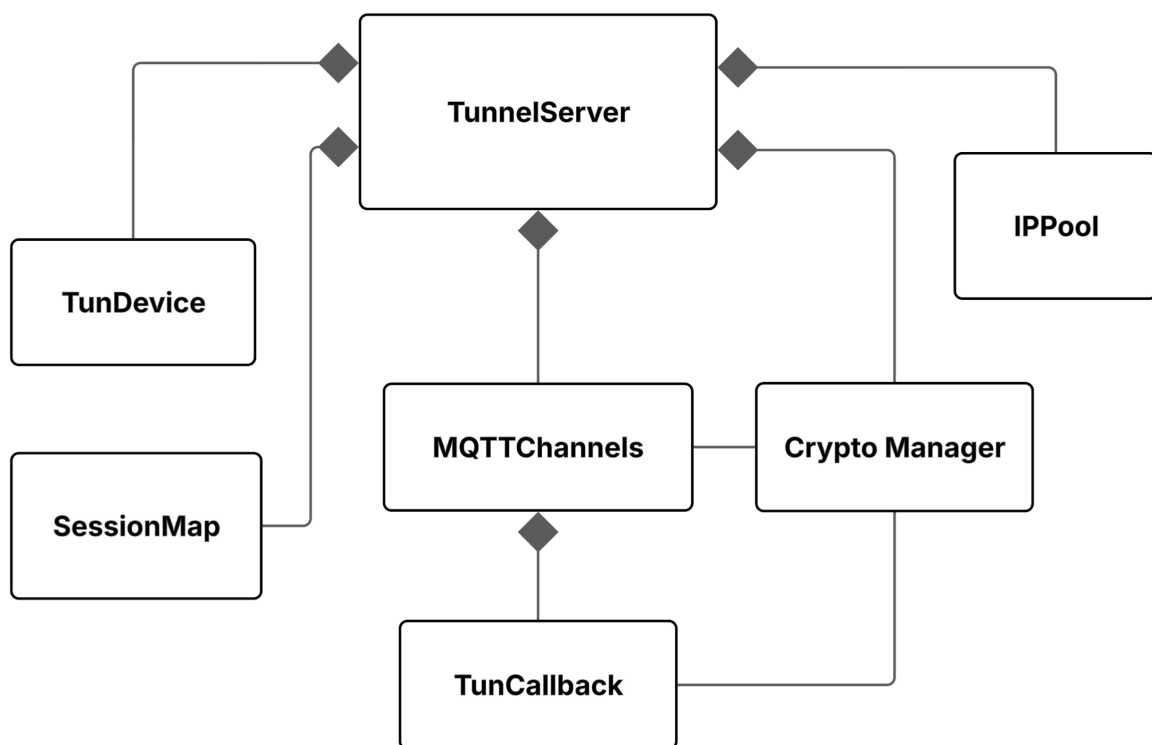


Abbildung 5 Kommunikationsmodell der Klasse TunnelServer

Beim Start konfiguriert der Server das TUN-Interface, aktiviert IP-Forwarding, verbindet sich mit dem MQTT Command- und Data-Channel und startet einen separaten Thread zum asynchronen Lesen von Paketen aus dem TUN-Device. Neue Clients werden über Nachrichten im Command-Channel über ein eigenes Handshake-Protokoll aufgenommen. Dabei wird pro Client dynamisch eine IP-Adresse aus dem IPPool vergeben, individuelle MQTT-Topics für Ein- und ausgehende Datenkanäle erzeugt und die Session Daten threadsafe in der SessionMap gespeichert.

Im Betrieb liest der Server IP-Pakete aus dem TUN-Interface, ermittelt anhand der Zieladresse die passende Client-Session und veröffentlicht die Daten über den zugehörigen MQTT-Topic. Optional werden Payloads pro Client mit dem integrierten CryptoManager verschlüsselt. Beim Stoppen wird der Thread beendet, MQTT-Verbindungen sauber getrennt und bei Session-Termination einzelner Clients Ressourcen wie IP-Adressen oder Routing/NAT-Regeln freigegeben.

Die Klasse TunnelClient implementiert einen MQTT-basierten Tunnel-Client, der Netzwerkpakete über ein lokales TUN-Interface liest und über MQTT an einen Tunnel-Server überträgt. Sie übernimmt analog zur TunnelServer Klasse den clientseitigen Aufbau, Betrieb und Abbau einer Tunnelsitzung inklusive Session-Handshake, Session-Konfiguration sowie optionaler Ende-zu-Ende-Verschlüsselung.

Beim Start (start_tunnel()) verbindet sich der Client mit dem Command-Channel eines TunnelServers, führt die Session-Aushandlung durch, verbindet anschließend den Data-Channel und startet einen separaten Thread, der kontinuierlich Daten vom TUN-Device liest und über MQTT veröffentlicht. Optional werden die Nutzdaten dabei über den integrierten CryptoManager verschlüsselt. Beim Stoppen (stop_tunnel()) wird der Tunnel sauber beendet, ein Termination-Message gesendet und die MQTT-Verbindungen werden geschlossen.

Die Klasse kapselt dabei zentrale Komponenten wie MQTTChannels, TunDevice und CryptoManager und stellt eine einfache API zur Steuerung des Tunnelbetriebs bereit. Der beschriebene Informationsaustausch wird in der folgenden Abbildung grafisch verdeutlicht.

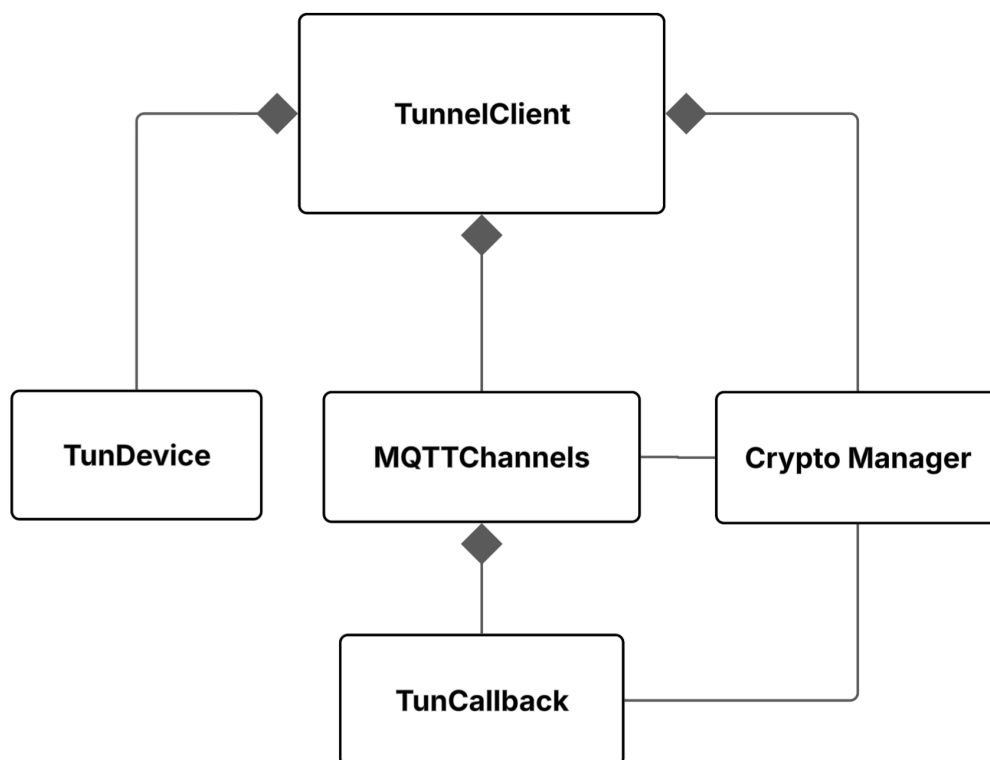


Abbildung 6 Kommunikationsmodell der Klasse TunnelClient

4.3 Installation

Um die Anwendung auf dem lokalen System betreiben zu können, müssen die folgenden Schritte durchgeführt werden.

1. Installieren der benötigten Pakete
2. Klonen des git repository auf einen Lokalen Zweig
3. Navigation der Shell in das zuvor geklonte Repository
4. Ausführen des Befehls `mkdir build && cd build`
5. Ausführen des Befehls `cmake ..`
6. Ausführen des Befehls `make`

Um einen korrekten Betrieb der Applikation zu gewährleisten müssen vor der Installation die folgenden Abhängigkeiten bzw. Pakete auf dem Zielsystem bereitgestellt werden.

- paho-MQTT C++
- GTest
- OpenSSL
- libsodium
- cereal
- spdlog
- fmt

Anschließend lässt sich die Anwendung über den Befehl aufrufen. Mittels des Parameters [Arguments] können verschiedene Betriebsmodi festgelegt werden. Die möglichen Argumente mit samt den dazugehörigen Erläuterungen sind der folgenden Tabelle aufgeführt.

Argument	Typ	Flag	Erforderlich	Beschreibung
Betriebsmodus	<str>	-m, --mode	Ja	Legt den Betriebsmodus des Tunnels fest (client, server)
Broker	<url>	-b, --broker	Ja	URI des MQTT-Brokers (z. B. tcp://localhost:1883)
Client-ID	<str>	-i, --client-id	Ja	Bezeichner des Clients
Command-Channel	<str>	-c, --command-channel	Ja	MQTT-Topic, das für den Tunnel-Setup/Handshake verwendet wird
Tunnel-Modus	<flag>	-v, --verbose	Nein	Tunnelmodus: einfache Verbindung oder Gateway-Modus. (connection / gateway)
Verbose-Modus	<flag>	-x, --insecure	Nein	Aktiviert ausführliche Log-Ausgaben für Debugging/Analyse
Verschlüsselung aus	<flag>	-t --ignore-server-auth	Nein	Deaktiviert die Verschlüsselung
Server-Auth aus	<str>	--interface	Nein	Client überspringt die Prüfung des Server-Public-Keys

Tabelle 2 Übersicht einzelner Parameter zum Programmaufruf

Dabei werden Parameter wie Betriebsmodus, Adresse des MQTT-Brokers, eine Client ID, MQTT-Topics oder das Versenden verschlüsselter Nachrichten eingestellt, bzw. zu- oder abgeschaltet.

```
sudo ./mqtt_tunnel [ARGUMENTS]
```

Ein möglicher Befehl kann demnach folgendermaßen aussehen.

```
./mqtt_tunnel -m server -b tcp://192.168.0.2:1883 -c tunnelServer -C tunnelServerCommand -e
```

Um einen korrekten Betrieb zu gewährleisten, muss der ausführende User über erweiterte Berechtigungen verfügen.

4.4 Programmablauf

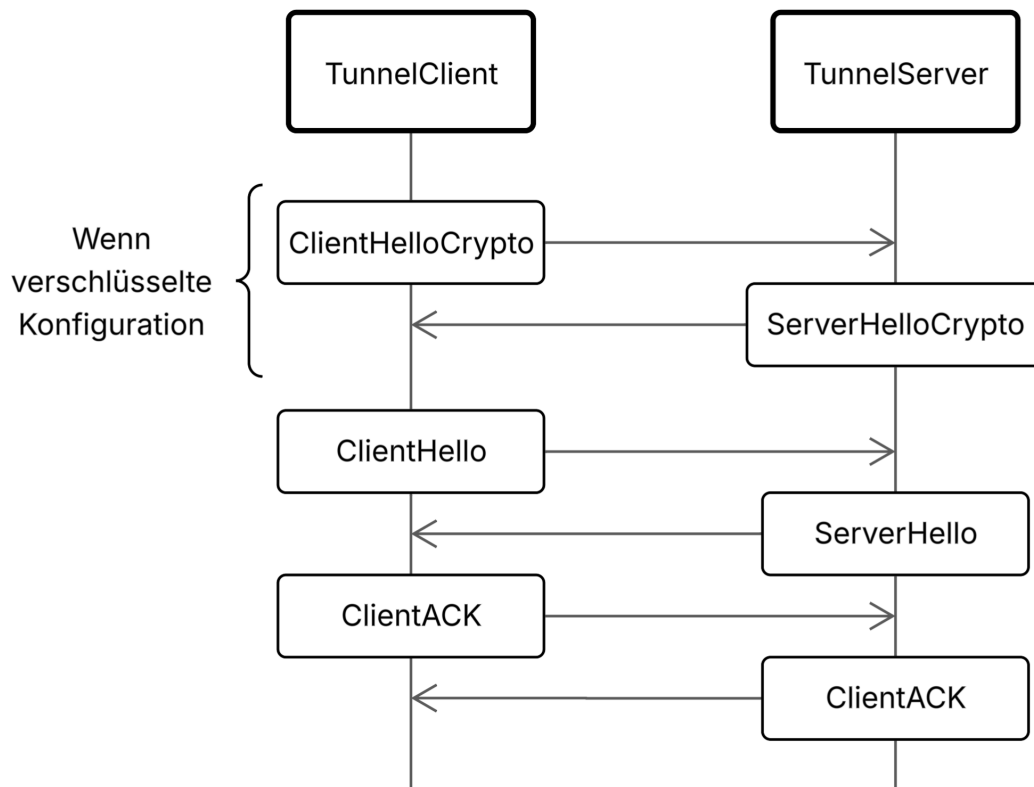


Abbildung 7 Verbindungsaufbau zwischen TunnelClient und TunnelServer

Der Verbindungsaufbau zwischen TunnelClient und TunnelServer erfolgt über den MQTT Command-Channel und dient dazu, eine Tunnelsession zu initialisieren. Die Abfolge des Informationsaustauschs ist in Abbildung 7 dargestellt. Während der Initialisierung werden folgende Ziele erreicht.

1. Identifikation des Clients
2. Aushandlung einer Session-ID
3. Zuweisung einer Client-IP-Adresse
4. Festlegung der MQTT Topics für Datentransfer
5. (Optional) Aufbau kryptographischer Session Keys für Verschlüsselung
6. Bestätigung der Session durch ACK-Nachrichten

Erst nach erfolgreichem Handshake wird der Data-Channel genutzt, um IP-Pakete über MQTT zu übertragen.

Die in Tabelle 3 aufgeführten Nachrichtentypen werden innerhalb der Implementierung unterschieden.

Nachrichtentyp	Kurzbeschreibung
CLIENT_HELLO_CRYPT	Startet den kryptographischen Handshake. Client startet x25519 Handshake, damit der Server eine sichere Session vorbereiten kann.
SERVER_HELLO_CRYPT O	Antwort auf den Crypto-Handshake. Enthält Serveranteil x25519 und eine Signatur und definiert eine eindeutige Client-Identifikation zur weiteren Kommunikation.
CLIENT_HELLO	Startet den eigentlichen Session-Handshake. Enthält Client-ID, handshake_identifizier und den gewünschten tunnel_mode.
SERVER_HELLO	Server bestätigt den Verbindungsaufbau und liefert Session-Parameter: zugewiesene Client-IP, Server-IP, MQTT-Topics und session_id.
CLIENT_ACK	Bestätigung des Clients, dass SERVER_HELLO empfangen und die Session-Konfiguration übernommen wurde.
SERVER_ACK	Abschlussbestätigung des Servers. Nach dieser Nachricht gilt die Session als vollständig etabliert.
ENCRYPTED_WRAPPER	Transporthülle für verschlüsselte Nachrichten. Enthält client_id zur Zuordnung und encrypted_payload als verschlüsselten Inhalt.
SESSION_TERMINATION	Beendet eine aktive Session. Server räumt Ressourcen auf

Tabelle 3 Überblick auftretender Nachrichtentypen

Dieses Handshake-Protokoll wurde implementiert, um eine möglichst dynamische Aushandlung der Session-Parameter zwischen Client und Server zu ermöglichen. Die Implementierung des MQTT-Tunnels basiert auf einer klaren Trennung zwischen Steuer- und Datenebene. Für den Verbindungsaufbau und die Sessionverwaltung wird ein separater MQTT Command-Channel genutzt, während der eigentliche Pakettransport über den MQTT Data-Channel erfolgt. Kontrollnachrichten (Handshake, Session-Termination) werden unabhängig von den getunnelten Daten verarbeitet. Die Topics für den Data-Channel werden pro Client dynamisch erzeugt und sind Bestandteil der ausgehandelten Session-Konfiguration. Auf Serverseite wird nach erfolgreichem Handshake das jeweilige Outbound-Topic des Clients abonniert, sodass eingehende Pakete eindeutig einer Session zugeordnet werden können. Auf diese Art und Weise wird die Zuordnung jedes Pakets zum jeweils verbundenen Client erlaubt und zum anderen durch den zentralen Command-Channel ein Single Point of Contact geschaffen über den die verbindenden Clients ihre Session-Konfiguration durchführen können.

Der Handshake ist als zustandsbasierter Ablauf implementiert und erlaubt es, Sessionparameter zur Laufzeit auszuhandeln. Dazu gehören die Vergabe einer freien Client-IP-Adresse aus dem IPPool, die Definition individueller MQTT Topics sowie die Generierung einer eindeutigen Session-ID.

Zur Vermeidung von Zustandskonflikten wird während des Handshakes pro Client ein temporärer Zustand gespeichert. Diese Zustände werden getrennt von den aktiven Sessions geführt und nach erfolgreichem Abschluss in die SessionMap übernommen.

Damit kann der Server mehrere parallele Verbindungsaufbauten unterstützen, ohne dass sich Handshake-Nachrichten verschiedener Clients gegenseitig beeinflussen. Der vollständige Ablauf des Handshakes ist in Abbildung 7 gezeigt und orientiert sich an gängigen Webprotokollen.

Für den Datentransfer werden IP-Pakete direkt aus dem TUN-Interface gelesen und als MQTT Payload transportiert. Auf Serverseite wird dazu der IPv4-Header des Pakets ausgewertet, um die Zieladresse zu extrahieren. Über diese Zieladresse erfolgt anschließend ein Lookup in der SessionMap, um die passende Session und damit das korrekte MQTT Topic für den Versand zu bestimmen. Dieses Verfahren ermöglicht ein transparentes Routing von Paketen zwischen mehreren Clients, ohne dass auf MQTT-Ebene zusätzliche Adressierungsinformationen erforderlich sind.

Zur Entkopplung von MQTT-Empfang und TUN-I/O wird der Paketfluss in beide Richtungen asynchron verarbeitet. Das Lesen aus dem TUN-Device erfolgt in einem separaten Thread, während der Command-Channel im Hauptthread ebenfalls non-blocking verarbeitet wird. Die Steuerung des Lebenszyklus erfolgt über atomare Flags, sodass ein kontrolliertes Stoppen des Servers bzw. Clients möglich ist. Zusätzlich werden MQTT Nachrichten im Command-Channel mit einem Retry-Mechanismus verarbeitet, um kurze Verzögerungen im Broker oder Netzwerk abzufangen bzw. bei Nichterreichbarkeit den Verbindungsaufbau automatisiert abubrechen.

Optional kann der Tunnel mit Ende-zu-Ende Verschlüsselung betrieben werden. In diesem Fall erfolgt vor dem eigentlichen Session-Handshake ein Schlüsselaustausch mittels X25519, wodurch pro Client ein separater kryptographischer Kontext etabliert wird. Die anschließende Datenübertragung erfolgt über AEAD-Verschlüsselung (256 Bit). Für den Transport verschlüsselter Inhalte des Command-Channels wird ein Wrapper-Format verwendet, das neben dem Ciphertext auch eine Client-Zuordnung enthält, damit der Empfänger den korrekten Schlüsselkontext auswählen kann. Der Lookup bei den Datenpaketen erfolgt über das eingehende MQTT-Topic, da dieses ebenfalls eine eindeutige Zuordnung erlaubt.

Diese Lösung wurde bewusst gegenüber reinem MQTT-over-TLS bevorzugt, da dadurch auch beim Einsatz öffentlicher Broker eine echte Ende-zu-Ende Absicherung zwischen TunnelClient und TunnelServer gewährleistet wird.

Beim ersten Start erzeugt der TunnelServer automatisch ein eigenes kryptographisches Schlüsselpaar und speichert den Public Key dauerhaft ab. Dadurch steht für nachfolgende Verbindungen eine stabile Server-Identität zur Verfügung, die vom TunnelClient zur Authentifizierung verwendet werden kann, sofern dieser PublicKey verteilt wird.

Der TunnelClient überprüft dabei standardmäßig die Identität des TunnelServers anhand des erwarteten und hinterlegten Public Keys. Diese Option lässt sich über das Flag

```
--no-server-auth
```

deaktivieren.

Für den Gatewaybetrieb werden zusätzlich systemseitige Netzwerkkonfigurationen vorgenommen. Der Server aktiviert IPv4-Forwarding und konfiguriert Forwarding-Regeln zwischen dem TUN-Interface und dem externen Netzwerkinterface. Im Gateway-Modus wird darüber hinaus eine NAT/Masquerading-Regel gesetzt, um ausgehenden Traffic der Clients über das Serverinterface ins externe Netz weiterzuleiten. Beim Beenden einer Session werden zugehörige Routen sowie NAT-Regeln entfernt und die vergebene IP-Adresse wieder in den Pool zurückgeführt.

Obwohl der Transportkanal des Tunnels (MQTT über TCP) eine zuverlässige und geordnete Übertragung gewährleistet, wurde die MTU des verwendeten TUN-Interfaces bewusst auf einen festen Wert begrenzt. Hintergrund ist, dass der Tunnel auf Paketebene arbeitet und IP-Pakete aus dem TUN-Device direkt als Payload in MQTT-Nachrichten kapselt. In der Implementierung werden diese IP-Pakete mit einem festen Puffer (z. B. 1500 Byte) aus dem TUN-Device gelesen und anschließend unverändert weitergeleitet. Würde das Betriebssystem IP-Pakete erzeugen, die größer als der verfügbare Puffer oder die erwartete MTU sind, könnte dies zu einer unvollständigen Paketübernahme (Trunkierung) und damit zu korrupten IP-Paketen führen. Solche Fehler lassen sich nicht durch TCP korrigieren, da TCP erst oberhalb der IP-Schicht wirkt und bereits beschädigte IP-Pakete vom Netzwerkstack verworfen werden.

Durch die explizite MTU-Begrenzung wird sichergestellt, dass der Kernel ausschließlich IP-Pakete innerhalb der erwarteten Größenordnung über das TUN-Interface ausliefert. Dies erhöht die Robustheit der Implementierung und reduziert das Risiko von Fragmentierung. Weiterhin wird MQTT QoS 1 verwendet, da es im Tunnelbetrieb eine ausreichende Zustellgarantie (mindestens einmal) bei moderatem Overhead bietet und damit mögliche Paketverluste deutlich reduziert, ohne die Latenz und Belastung des Brokers wie bei QoS 2 durch einen 4-way-Handshake unnötig zu erhöhen.

4.5 Konfiguration und Steuerung

Die Steuerung der Tunnel-Software erfolgt über verschiedene Argumente, die den Betriebsmodus und die Sicherheitsmaßnahmen definieren. Diese Argumente wurden bereits unter 4.3 in Tabelle 2 *Übersicht einzelner Parameter zum Programmaufruf* aufgeführt.

Für den Verbindungsaufbau sind zunächst die Rolle bzw. der Betriebsmodus¹⁰ sowie die URI¹¹ des MQTT-Brokers erforderlich. Jeder Teilnehmer identifiziert sich über eine eindeutige Client-ID, die für die Client-Rolle auch als Basis für die individuelle Benennung der Data-Topics dient. Die Bezeichnung des Command-Topics muss zwischen Server und verbindenden Clients übereinstimmen.

Im Netzwerkbereich kann zwischen zwei Tunnel-Modi gewählt werden.

- Punkt-zu-Punkt-Verbindung
- Gateway-Modus

Bei der Punkt-zu-Punkt-Verbindung handelt es sich um eine Direktverbindung zwischen den Teilnehmern im klassischen Sinne.

Der Gateway-Modus erfordert auf der Server-Seite die Angabe einer Netzwerkschnittstelle (Interface), um die notwendigen NAT-Regeln für die Traffic-Weiterleitung korrekt anzuwenden. Die Default-Route des Client-Hosts wird über den Server gelegt.

Die Verschlüsselung des Datentransfers ist standardmäßig aktiv, kann jedoch über das Flag

```
--insecure
```

deaktiviert werden.

¹⁰ festgelegt als Client oder Server

¹¹ Uniform Resource Identifier

Zusätzlich bietet die Option

`--no-server-auth`

die Möglichkeit, die Prüfung des Server-Public-Keys zu überspringen, was die Flexibilität in Testumgebungen erhöht. Für Diagnosezwecke steht ein Verbose-Modus¹² zur Verfügung, der ausführliche Log-Ausgaben für die Fehlersuche generiert.

Abbildung 8 dient der Verdeutlichung zum Kommunikationsfluss zwischen einzelner Clients und dem einbezogenen MQTT-Broker.

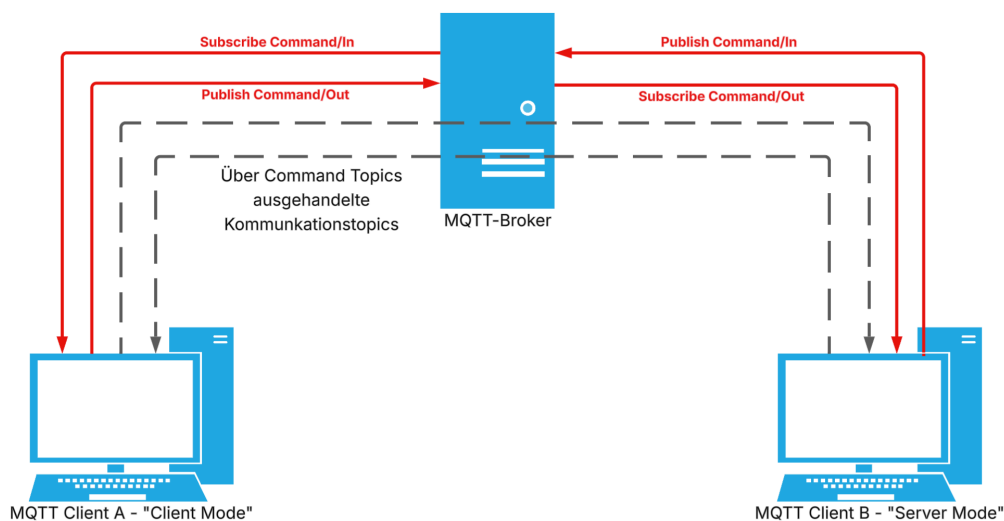


Abbildung 8 Kommunikationsprinzip zwischen der MQTT-Clients

5. Tests und Analyse

In diesem Kapitel werden die verschiedenen Testszenarien beleuchtet und dargestellt. Dabei werden sowohl die individuell verfolgten Zielen mit samt verwendeter Befehle aufgeführt. Die entsprechenden Ergebnisse in Form von Terminalausgaben liegen im Anhang dieser Ausarbeitung bei.

5.1 Funktionale Tests

Zur Überprüfung der grundlegenden Funktionsfähigkeit des MQTT-Tunnels wurden mehrere funktionale Tests durchgeführt. Ziel dieser Tests war es, nachzuweisen, dass TCP-basierte Kommunikation erfolgreich über den MQTT-basierten Tunnel übertragen werden kann und für die beteiligten Anwendungen transparent funktioniert.

5.1.1 Aufbau des Testszenarios

¹² angelehnt aus dem Englischen *verbose* - wortreich, weitschweifig

Der Tunnel-Server wurde auf der virtuellen Maschine **mqtt-vm-a** gestartet, der Tunnel-Client auf **mqtt-vm-b**. Nach erfolgreichem Verbindungsaufbau waren auf beiden Systemen jeweils ein TUN-Interface (**tun0**) mit folgenden IP-Adressen aktiv.

- **Tunnel-Server (mqtt-vm-a):** 10.0.0.1/24
- **Tunnel-Client (mqtt-vm-b):** 10.0.0.2/24

Die Verbindung wurde zuvor über den MQTT-Broker erfolgreich etabliert, einschließlich des vollständigen Handshakes zwischen Client und Server.

Test 1: Erreichbarkeit über den Tunnel (Ping)

Als erster Funktionstest wurde die grundlegende Erreichbarkeit über den Tunnel überprüft. Dazu wurde vom Tunnel-Client ein ICMP-Ping an die Tunnel-IP des Servers gesendet.

```
Ping 10.0.0.1
```

Die ICMP-Pakete wurden erfolgreich übertragen und beantwortet, was bestätigt, dass IP-Verkehr korrekt über das TUN-Interface gelesen, über MQTT transportiert und auf der Gegenseite wieder in das Netzwerk eingespeist wurde.

Test 2: TCP-Kommunikation über den Tunnel

Im nächsten Schritt wurde überprüft, ob auch TCP-basierte Kommunikation zuverlässig über den Tunnel funktioniert. Auf dem Tunnel-Server wurde ein einfacher TCP-Listener mit **netcat** gestartet.

```
nc -l -p 12345
```

Anschließend wurde vom Tunnel-Client aus eine TCP-Verbindung über die Tunnel-IP aufgebaut und eine Testnachricht gesendet.

```
printf '%s\n' 'TESTNACHRICHT' | nc 10.0.0.1 12345
```

Die Testnachricht wurde auf der Serverseite korrekt empfangen und angezeigt. Damit konnte nachgewiesen werden, dass TCP-Verbindungen vollständig über den MQTT-Tunnel übertragen werden können, ohne dass die beteiligten Anwendungen angepasst werden müssen.

5.1.2 Ergebnis der funktionalen Tests

Die durchgeführten Tests zeigen, dass der implementierte MQTT-Tunnel in der Lage ist, sowohl ICMP- als auch TCP-Verkehr zuverlässig zu transportieren. Für die Anwendungen erscheint die Verbindung wie eine normale IP-basierte Punkt-zu-Punkt-Verbindung, während der eigentliche Datentransport vollständig über MQTT erfolgt. Damit ist die grundlegende Funktionalität des Tunnels nachgewiesen.

5.2 Performance und Latenz

Zur Bewertung der Leistungsfähigkeit des MQTT-basierten Tunnels wurden Durchsatz- und Latenztests mit *iperf3* durchgeführt. Die Messungen erfolgten über das virtuelle Tunnel-Interface (**tun0**) zwischen Tunnel-Client (**10.0.0.2**) und Tunnel-Server (**10.0.0.1**). Zusätzlich wurden Referenzmessungen ohne Tunnel über das Host-Only-Netzwerk durchgeführt.

Neben dem reinen Durchsatz wurde bei den UDP-Messungen zusätzlich der Jitter betrachtet. Jitter beschreibt die zeitliche Schwankung zwischen aufeinanderfolgenden Paketen und stellt damit ein Maß für die Stabilität der Paketlaufzeiten dar. Ein niedriger Jitter weist auf eine gleichmäßige und vorhersehbare Übertragung hin, während ein hoher Jitter auf Verzögerungsschwankungen im Übertragungspfad schließen lässt. Insbesondere bei zeitkritischen Anwendungen kann ein erhöhter Jitter die Übertragungsqualität negativ beeinflussen.

5.2.1 TCP-Durchsatz

Im **Baseline-Betrieb (ohne Verschlüsselung)** wurde bei TCP-Messungen ein Durchsatz von etwa **8,7 Mbit/s** im Client-zu-Server-Betrieb erreicht. Im Reverse-Betrieb (Server zu Client) lag der Durchsatz bei etwa **2,6 Mbit/s**. Die Messergebnisse zeigten dabei eine deutliche Abhängigkeit von der verwendeten TCP-Segmentgröße. Kleinere Segmentgrößen führten zu einem signifikanten Rückgang des Durchsatzes, während größere Segmente höhere Übertragungsraten ermöglichten.

Im **verschlüsselten Betrieb** lag der TCP-Durchsatz im Client-zu-Server-Betrieb bei etwa **5,9 Mbit/s**. Im Reverse-Betrieb wurden Werte von etwa **2,6–2,7 Mbit/s** gemessen. Damit zeigte sich im Forward-Betrieb ein moderater Rückgang des Durchsatzes gegenüber dem unverschlüsselten Modus, während im Reverse-Betrieb kein signifikanter Unterschied feststellbar war.

5.2.2 UDP-Durchsatz, Jitter und Paketverlust

UDP-Messungen zeigten sowohl im unverschlüsselten als auch im verschlüsselten Betrieb bis zu einer Datenrate von **5 Mbit/s** einen stabilen Betrieb ohne Paketverluste. Der gemessene Jitter lag dabei im Bereich von etwa **1–2 ms**.

Bei einer Zielrate von **10 Mbit/s** traten in beiden Betriebsarten Paketverluste auf. Im unverschlüsselten Betrieb wurde ein effektiver Durchsatz von etwa **5,9 Mbit/s** bei einem Paketverlust von rund **38 %** gemessen. Im verschlüsselten Betrieb lag der effektive Durchsatz bei etwa **7,4 Mbit/s**, wobei der Paketverlust bei etwa **23 %** lag.

5.2.3 Referenzmessungen ohne Tunnel

Die Referenzmessungen ohne Tunnel über das Host-Only-Netzwerk zeigten deutlich höhere Leistungswerte. Hier wurden TCP-Durchsatzraten von etwa **2,6–2,7 Gbit/s** erreicht. UDP-Messungen mit **10 Mbit/s** verliefen ohne Paketverlust und mit einem Jitter von unter **0,1 ms**.

5.2.4 Einordnung der Ergebnisse

Die Messergebnisse zeigen, dass die Performance des Tunnels primär durch das MQTT-basierte Transportmodell begrenzt wird. Insbesondere die Kapselung jedes IP-Pakets in einzelne MQTT-Nachrichten führt zu einem hohen Overhead und begrenzt den maximal erreichbaren Durchsatz. Die zusätzliche Ende-zu-Ende-Verschlüsselung verursacht im Vergleich dazu lediglich einen moderaten Mehraufwand und stellt keinen dominanten limitierenden Faktor dar.

5.3 Analyse mittels tcpdump

Zur Analyse der Sichtbarkeit des MQTT-Tunnels aus Sicht eines externen Beobachters wurden Netzerkafzeichnungen mit **tcpdump** durchgeföhrt. Ziel war es, zu untersuchen, welche Art von Netzwerkverkehr auf den beteiligten Schnittstellen sichtbar ist und ob der transportierte TCP-Verkehr als solcher erkennbar bleibt.

5.3.1 Analyse auf dem TUN-Interface

Zunächst wurde der Netzwerkverkehr direkt auf dem TUN-Interface (**tun0**) des Tunnel-Servers aufgezeichnet. Auf dieser Schnittstelle sind die durch den Tunnel transportierten IP-Pakete sichtbar, da sie hier vom Kernel an die Tunnelanwendung übergeben bzw. von dieser wieder eingespeist werden.

Die Aufzeichnung zeigt unter anderem den vollständigen TCP-Verbindungsaufbau (SYN, SYN-ACK, ACK) sowie die Übertragung der Nutzdaten. Die Testnachricht „TESTNACHRICHT“ wird dabei als Teil eines regulären TCP-Datenpakets sichtbar. Dies bestätigt, dass innerhalb des Tunnels echter TCP-Verkehr transportiert wird und der Tunnel aus Sicht des Betriebssystems wie eine normale IP-Verbindung funktioniert.

5.3.2 Analyse auf der physischen Netzwerkschnittstelle (ohne MQTT)

Anschließend wurde der Netzwerkverkehr auf der physischen Netzwerkschnittstelle (**enp0s8**) des Tunnel-Servers aufgezeichnet, wobei MQTT-Verkehr explizit ausgeschlossen wurde (**not port 1883**). In dieser Aufzeichnung waren keine Pakete sichtbar, die auf eine direkte TCP-Verbindung zwischen Tunnel-Client und Tunnel-Server hindeuten. Insbesondere war weder die Testnachricht noch ein TCP-Handshake in Richtung der Tunnel-IP-Adressen (**10.0.0.0/24**) erkennbar. Für einen Beobachter auf dieser Schnittstelle ist somit kein Hinweis auf die tatsächlich durch den Tunnel transportierte TCP-Kommunikation vorhanden.

5.3.3 Analyse des MQTT-Verkehrs

In einem weiteren Schritt wurde gezielt der MQTT-Verkehr auf Port 1883 aufgezeichnet. In dieser Aufzeichnung sind ausschließlich TCP-Verbindungen zwischen Tunnel-Client, Tunnel-Server und dem MQTT-Broker sichtbar. Die übertragenen Daten erscheinen dabei als regulärer MQTT-Datenverkehr mit Payloads unterschiedlicher Länge.

Der Inhalt der MQTT-Nachrichten lässt ohne zusätzliche Kontextinformationen keinen direkten Rückschluss auf den darin gekapselten TCP-Verkehr zu. Insbesondere sind weder Zieladressen noch Portnummern der eigentlichen TCP-Verbindungen auf Netzwerkebene unmittelbar erkennbar.

5.3.4 Ergebnis der Analyse

Die Analyse zeigt deutlich, dass der eigentliche TCP-Verkehr ausschließlich auf dem TUN-Interface sichtbar ist. Auf der physischen Netzwerkschnittstelle ist für einen externen Beobachter lediglich MQTT-Kommunikation zwischen den beteiligten Systemen und dem Broker erkennbar.

Damit erfüllt der Tunnel das Ziel, TCP-Verkehr effektiv in MQTT zu kapseln und auf Netzwerkebene zu verbergen. Ein Dritter kann zwar feststellen, dass MQTT genutzt wird, jedoch nicht ohne weiteres erkennen, welche Art von Anwendungen oder Protokollen innerhalb des Tunnels transportiert werden.

5.4 Fehlerfälle

Neben den erfolgreichen Funktionstests wurden auch verschiedene Fehlerfälle betrachtet, um das Verhalten des MQTT-Tunnels bei nicht idealen Bedingungen zu analysieren. Ziel war es zu überprüfen, wie sich der Tunnel bei fehlender Verbindung, nicht verfügbaren Diensten oder unterbrochenem Tunnelbetrieb verhält.

Fehlerfall 1: TCP-Verbindung ohne aktiven Tunnel

Wird versucht, vom Tunnel-Client aus eine TCP-Verbindung zur Tunnel-IP des Servers aufzubauen, ohne dass der Tunnel-Server aktiv ist, schlägt der Verbindungsaufbau fehl. Der Client erhält in diesem Fall eine Timeout- oder Verbindungsfehler-Meldung, da keine Gegenstelle erreichbar ist.

Dieses Verhalten entspricht dem einer normalen IP-Verbindung ohne erreichbaren Zielhost und zeigt, dass der Tunnel keine unerwarteten Fallback-Mechanismen verwendet.

Fehlerfall 2: Ping bei nicht aktivem Tunnel

Ein ICMP-Ping an die Tunnel-IP-Adresse des Servers (**10.0.0.1**) führt bei nicht aktivem Tunnel zu einem vollständigen Paketverlust. Es werden keine ICMP-Antworten empfangen. Auch dieses Verhalten entspricht dem erwarteten Verhalten einer nicht vorhandenen oder unterbrochenen Netzwerkverbindung und bestätigt, dass der Tunnelverkehr ausschließlich über den aktiven MQTT-Tunnel abgewickelt wird.

Fehlerfall 3: Unterbrochene TCP-Sitzung

Wird eine bestehende TCP-Verbindung über den Tunnel unterbrochen, beispielsweise durch das Beenden des TCP-Servers oder des Tunnel-Servers, schlägt ein erneuter Verbindungsversuch fehl. Der Client erhält in diesem Fall eine entsprechende Fehlermeldung (z. B. „Connection refused“).

Der Tunnel verhält sich hierbei transparent: Die Fehlermeldungen stammen aus dem regulären TCP/IP-Stack des Betriebssystems und werden unverändert an die Anwendung weitergereicht.

Zusammenfassung der Fehlerfälle

Die untersuchten Fehlerfälle zeigen, dass sich der MQTT-Tunnel bei Störungen oder Fehlkonfigurationen wie eine reguläre IP-basierte Verbindung verhält. Fehler werden nicht verborgen oder abgefangen, sondern korrekt an die jeweiligen Anwendungen weitergegeben.

Dies erleichtert sowohl die Fehlersuche als auch die Bewertung des Tunnelverhaltens und trägt zu einem transparenten und nachvollziehbaren Systemverhalten bei.

6. Fazit

Im Rahmen dieses Projekts wurde ein IP-basierter Netzwerk-Tunnel implementiert, der TCP- und ICMP-Verkehr vollständig über MQTT transportiert. Ziel war es, auf der physischen Netzwerkschnittstelle ausschließlich MQTT-Traffic sichtbar zu machen und dennoch eine transparente IP-Kommunikation zwischen Client und Server zu ermöglichen.

Die funktionalen Tests bestätigten die korrekte Arbeitsweise des Tunnels. IP-basierte Kommunikation war ausschließlich über das virtuelle TUN-Interface möglich, während auf der physischen Schnittstelle nur MQTT-Verkehr beobachtet wurde. Eine Analyse mittels `tcpdump` zeigte, dass keine IP- oder TCP-Nutzdaten außerhalb des Tunnels sichtbar waren.

Die durchgeführten Performance- und Latenztests zeigten, dass der erreichbare Durchsatz des Tunnels deutlich unterhalb der nativen Netzwerkleistung liegt. Der maximale TCP-Durchsatz bewegte sich im Bereich weniger Mbit/s und war stark von der effektiven Paket- und Segmentgröße abhängig. UDP-Messungen zeigten bei höheren Datenraten einen zunehmenden Paketverlust. Diese Einschränkungen sind auf das MQTT-basierte Transportmodell und den damit verbundenen Nachrichten-Overhead zurückzuführen.

Die optionale Ende-zu-Ende-Verschlüsselung auf Anwendungsebene wurde erfolgreich integriert. Ein Vergleich zwischen unverschlüsseltem und verschlüsseltem Betrieb zeigte, dass die zusätzliche kryptografische Verarbeitung keinen dominanten Einfluss auf die Gesamtperformance hat. Die Wirksamkeit der Verschlüsselung konnte durch eine Analyse des aufgezeichneten MQTT-Datenverkehrs nachgewiesen werden, bei der im verschlüsselten Betrieb keine Klartext-Nutzdaten mehr identifizierbar waren.

Zusammenfassend zeigt das Projekt, dass MQTT grundsätzlich als Transportmechanismus für IP-Verkehr verwendet werden kann, jedoch mit deutlichen Einschränkungen hinsichtlich der Übertragungsleistung. Der entwickelte Tunnel eignet sich damit vor allem für Szenarien mit geringen Datenraten oder speziellen Anforderungen an Protokollkapselung und Sichtbarkeit, weniger jedoch für leistungsintensive Netzwerkkommunikation.

Abbildungsverzeichnis

Abbildung 1 Übersicht MQTT-Clients und MQTT-Broker	9
Abbildung 2 Entwurf zum initial angestrebten Funktionsablaufs	10
Abbildung 3 Übersicht zum Informationsfluss der MQTT-Topics	10
Abbildung 4 Auszug der Sourcefiles aus dem Github Repository	17
Abbildung 5 Kommunikationsmodell der Klasse TunnelServer	18
Abbildung 6 Kommunikationsmodell der Klasse TunnelClient	19
Abbildung 7 Verbindungsaufbau zwischen TunnelClient und TunnelServer	21
Abbildung 8 Kommunikationsprinzip zwischen der MQTT-Clients	25

Tabellenverzeichnis

Tabelle 1 Übersicht der Quality of Service Level	12
Tabelle 2 Übersicht einzelner Parameter zum Programmaufruf	20
Tabelle 3 Überblick auftretender Nachrichtentypen	22

Quellenverzeichnis

1. https://docs.oracle.com/cd/A97335_02/apps.102/a83722/overvie3.htm#1014532
2. <https://www.ibm.com/docs/en/app-connect/11.0.0?topic=corba-common-object-request-broker-architecture>
3. https://refubium.fu-berlin.de/bitstream/handle/fub188/4102/12_chap12.pdf
4. <https://learn.microsoft.com/de-de/dotnet/standard/native-interop/type-marshalling>
5. https://www.ois-com.translate.goog/index.php/about-corba/?_x_tr_sl=en&_x_tr_tl=de&_x_tr_hl=de&_x_tr_pto=sge#:~:text=CORBA%20is%20the%20world's%20leading,programming%20languages%2C%20and%20operating%20systems.
6. [https://www.elpassion.com/de/glossary/what-is-common-object-request-broker-architecture-corba#:~:text=Eine%20der%20Hauptfunktionen%20von%20CORBA%20\(%20Common,zwischen%20verschiedenen%20Plattformen%20und%20Programmiersprachen%20zu%20unterst%C3%BCtzen.](https://www.elpassion.com/de/glossary/what-is-common-object-request-broker-architecture-corba#:~:text=Eine%20der%20Hauptfunktionen%20von%20CORBA%20(%20Common,zwischen%20verschiedenen%20Plattformen%20und%20Programmiersprachen%20zu%20unterst%C3%BCtzen.)
7. <https://www.omg.org/corba/faq.htm>
8. <https://hilfe.comarch.de/cee/index.php/documentation/corba-schnittstelle/>
9. <https://www.arl.wustl.edu/Publications/2000-04/comp00fgk.pdf>
10. <https://umu.diva-portal.org/smash/get/diva2:1987951/FULLTEXT01.pdf>
11. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
12. <https://iot.telekom.com/de/blog/mqtt-protokoll-funktionsweise-anwendungen-und-vorteile-im-iot>
13. <https://www.opc-router.de/was-ist-mqtt/>
14. https://aws.amazon-com.translate.goog/what-is/mqtt/?_x_tr_sl=en&_x_tr_tl=de&_x_tr_hl=de&_x_tr_pto=sge#:~:text=Smart%20sensors%2C%20wearables%2C%20and%20other,can%20communicate%20IoT%20data%20efficiently.
15. <https://www.all-electronics.de/automatisierung/basiswissen-mqtt-was-kann-das-iot-kommunikations-protokoll/700581#:~:text=Antwort%205:%20MQTT%20verwendet%20QoS%2DModi%2C%20um%20die,QoS%2DStufen:%200%2C%201%20und%202%2C%20die%20verschiedene>
16. <https://www.pubnub.com/blog/what-is-mqtt-use-cases/>
17. <https://www.ibm.com/docs/de/ibm-mq/9.3.x?topic=overview-mq-telemetry-transport-protocol>
18. <http://www.bbs-1.de/bbs1/umat/netze/netz8.html>
19. <https://www.kernel.org/doc/html/latest/networking/tuntap.html>
20. https://www.researchgate.net/publication/317391853_IoT_real_time_data_acquisition_using_MQTT_protocol
21. [Evaluation of MQTT Bridge Architectures in a Cross-Organizational Context von Keila Lima, Tosin Daniel Oyetoyan, Rogardt Heldal, Wilhelm Hasselbring aus 01/2025](#)
22. <https://www.emqx.com/en/blog/mqtt5-new-features-payload-format-indicator-and-content-type>
23. <https://mymqtt.app/de/mqtt-qos-levels>
- 24.

Anhang

Anhang A: Terminalausgaben – Tunnelbetrieb

A1: Start des MQTT-Tunnel-Servers (mqtt-vm-a)

```
usera@mqtt-vm-a:~$ cd ~/MQTT_Tunnel/build
sudo ./mqtt_tunnel \
  --mode server \
  --broker 192.168.56.101 \
  --client-id tunnel-server-a
[sudo] password for usera:
[2025-12-21 18:51:04.937] [info] Creating TUN device: tun0
[2025-12-21 18:51:04.942] [info] TUN device tun0 created
[2025-12-21 18:51:04.979] [info] TUN device configured with IP: 10.0.0.1
[2025-12-21 18:51:04.979] [info] Server: Connecting to command channel...
[2025-12-21 18:51:05.000] [info] Server: Connected! Subscribing to topic: mqtt_tunnel/
commands_RX
[2025-12-21 18:51:05.003] [info] Server: Subscribed successfully
[2025-12-21 18:51:05.003] [info] Command channel connected
Data channel connected
[2025-12-21 18:51:06.005] [info] Tunnel server started
[2025-12-21 18:51:06.006] [warning] No active session for destination IP: 96.164.82.55
[2025-12-21 18:51:09.556] [warning] No active session for destination IP: 96.164.82.55
[2025-12-21 18:51:12.616] [info] Consumed message from command channel
[2025-12-21 18:51:12.617] [info] Handling client handshake message
[2025-12-21 18:51:12.617] [info] Received Client Hello from client ID: tunnel-client-b
[2025-12-21 18:51:12.660] [info] Sent Server Hello to client ID: tunnel-client-b
[2025-12-21 18:51:12.709] [info] Received Client ACK from client ID: tunnel-client-b
[2025-12-21 18:51:12.752] [info] Sent Server ACK to client ID: tunnel-client-b
[2025-12-21 18:51:12.767] [info] Session established for client ID: tunnel-client-b with IP:
10.0.0.2
[2025-12-21 18:51:14.858] [info] Message arrived with topic: mqtt_tunnel/commands/tunnel-
client-b/B
[2025-12-21 18:51:14.858] [info] Wrote 48 bytes to TUN device
[2025-12-21 18:51:17.270] [info] Message arrived with topic: mqtt_tunnel/commands/tunnel-
client-b/B
[2025-12-21 18:51:17.270] [info] Wrote 48 bytes to TUN device
[2025-12-21 18:51:18.772] [warning] No active session for destination IP: 96.164.82.55
[2025-12-21 18:51:26.230] [info] Message arrived with topic: mqtt_tunnel/commands/tunnel-
client-b/B
[2025-12-21 18:51:26.231] [info] Wrote 48 bytes to TUN device
[2025-12-21 18:51:37.717] [warning] No active session for destination IP: 96.164.82.55
[2025-12-21 18:51:44.674] [info] Message arrived with topic: mqtt_tunnel/commands/tunnel-
client-b/B
[2025-12-21 18:51:44.675] [info] Wrote 48 bytes to TUN device
[2025-12-21 18:51:56.810] [info] Message arrived with topic: mqtt_tunnel/commands/tunnel-
client-b/B
[2025-12-21 18:51:56.811] [info] Read 60 bytes from TUN and publishing to topic
mqtt_tunnel/commands/tunnel-client-b/A
```

[2025-12-21 18:51:56.812] [info] Wrote 60 bytes to TUN device
[2025-12-21 18:51:56.860] [info] Message arrived with topic: mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:51:56.862] [info] Wrote 52 bytes to TUN device
[2025-12-21 18:51:56.863] [info] Message arrived with topic: mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:51:56.864] [info] Read 52 bytes from TUN and publishing to topic mqtt_tunnel/commands/tunnel-client-b/A
[2025-12-21 18:51:56.864] [info] Wrote 66 bytes to TUN device
[2025-12-21 18:52:12.532] [warning] No active session for destination IP: 96.164.82.55
[2025-12-21 18:52:21.527] [info] Message arrived with topic: mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:52:21.527] [info] Wrote 48 bytes to TUN device
[2025-12-21 18:53:07.317] [info] Message arrived with topic: mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:53:07.318] [info] Wrote 52 bytes to TUN device
[2025-12-21 18:53:07.319] [info] Read 52 bytes from TUN and publishing to topic mqtt_tunnel/commands/tunnel-client-b/A
[2025-12-21 18:53:07.364] [info] Message arrived with topic: mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:53:07.365] [info] Wrote 52 bytes to TUN device
[2025-12-21 18:53:08.279] [info] Message arrived with topic: mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:53:08.280] [info] Read 40 bytes from TUN and publishing to topic mqtt_tunnel/commands/tunnel-client-b/A
[2025-12-21 18:53:08.281] [info] Wrote 60 bytes to TUN device
[2025-12-21 18:53:24.212] [warning] No active session for destination IP: 96.164.82.55
[2025-12-21 18:53:39.353] [info] Message arrived with topic: mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:53:39.353] [info] Wrote 48 bytes to TUN device

B1: Start des MQTT-Tunnel-Clients (mqtt-vm-b)

```
userb@mqtt-vm-b:~$ cd ~/MQTT_Tunnel/build
sudo ./mqtt_tunnel \
  --mode client \
  --broker 192.168.56.101 \
  --client-id tunnel-client-b
[sudo] password for userb:
[2025-12-21 18:51:12.413] [info] Creating TUN device: tun0
[2025-12-21 18:51:12.418] [info] TUN device tun0 created
[2025-12-21 18:51:12.419] [info] Connecting to command channel...
[2025-12-21 18:51:12.532] [info] Connected! Now subscribing to topic: mqtt_tunnel/
commands_TX
[2025-12-21 18:51:12.534] [info] Subscribed successfully
[2025-12-21 18:51:12.535] [info] Command channel connected
[2025-12-21 18:51:12.537] [info] Sent Client Hello, waiting for Server Hello...
[2025-12-21 18:51:12.667] [info] Received Server Hello
[2025-12-21 18:51:12.668] [info] Session configured with Client ID: tunnel-client-b, IP:
10.0.0.2, ServerIP: 10.0.0.1 Inbound Topic: mqtt_tunnel/commands/tunnel-client-b/A,
Outbound Topic: mqtt_tunnel/commands/tunnel-client-b/B, Session ID:
4a7a39dff2e420e2e52a6c252f30ae7562bec4ac5fec44f4f3749586275a3d38
[2025-12-21 18:51:12.758] [info] Received Server ACK, session handshake complete
[2025-12-21 18:51:12.759] [info] Configuring TUN device with IP and routes...
[2025-12-21 18:51:12.759] [info] Executing command: ip addr add 10.0.0.2/24 dev tun0
[2025-12-21 18:51:12.760] [info] Executing command: ip route add 10.0.0.1 dev tun0
[2025-12-21 18:51:12.813] [info] TUN device configured with IP: 10.0.0.2
[2025-12-21 18:51:12.821] [info] Route to server address 10.0.0.1 added
[2025-12-21 18:51:14.862] [info] Data channel connected
[2025-12-21 18:51:14.863] [info] Tunnel client started
[2025-12-21 18:51:14.864] [info] Read 48 bytes from TUN and published to topic
mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:51:17.277] [info] Read 48 bytes from TUN and published to topic
mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:51:26.236] [info] Read 48 bytes from TUN and published to topic
mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:51:44.675] [info] Read 48 bytes from TUN and published to topic
mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:51:56.791] [info] Read 60 bytes from TUN and published to topic
mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:51:56.820] [info] Message arrived with topic: mqtt_tunnel/commands/tunnel-
client-b/A
[2025-12-21 18:51:56.820] [info] Wrote 60 bytes to TUN device
[2025-12-21 18:51:56.866] [info] Read 52 bytes from TUN and published to topic
mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:51:56.867] [info] Read 66 bytes from TUN and published to topic
mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:51:56.910] [info] Message arrived with topic: mqtt_tunnel/commands/tunnel-
client-b/A
[2025-12-21 18:51:56.910] [info] Wrote 52 bytes to TUN device
[2025-12-21 18:52:21.533] [info] Read 48 bytes from TUN and published to topic
mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:53:07.323] [info] Read 52 bytes from TUN and published to topic
mqtt_tunnel/commands/tunnel-client-b/B
```

```
[2025-12-21 18:53:07.327] [info] Message arrived with topic: mqtt_tunnel/commands/tunnel-client-b/A
[2025-12-21 18:53:07.328] [info] Wrote 52 bytes to TUN device
[2025-12-21 18:53:07.371] [info] Read 52 bytes from TUN and published to topic
mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:53:08.285] [info] Read 60 bytes from TUN and published to topic
mqtt_tunnel/commands/tunnel-client-b/B
[2025-12-21 18:53:08.288] [info] Message arrived with topic: mqtt_tunnel/commands/tunnel-client-b/A
[2025-12-21 18:53:08.288] [info] Wrote 40 bytes to TUN device
[2025-12-21 18:53:39.361] [info] Read 48 bytes from TUN and published to topic
mqtt_tunnel/commands/tunnel-client-b/B
```

A2: TCP-Listener mit netcat (mqtt-vm-a)

```
Last login: Sun Dec 21 18:32:46 2025 from 192.168.56.1
usera@mqtt-vm-a:~$ nc -l -p 12345
TESTNACHRICHT
usera@mqtt-vm-a:~$
```

B2: TCP-Client mit netcat (mqtt-vm-b)

```
userb@mqtt-vm-b:~$ printf '%s\n' 'TESTNACHRICHT' | nc 10.0.0.1 12345
```

A3: tcpdump auf dem Host-Only-Interface – ohne MQTT (mqtt-vm-a, Interface: enp0s8, Filter: not port 1883)

```
usera@mqtt-vm-a:~$ sudo tcpdump -i enp0s8 -n -c 20 not port 1883
[sudo] password for usera:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp0s8, link-type EN10MB (Ethernet), snapshot length 262144 bytes
18:51:41.387920 IP 192.168.56.101.22 > 192.168.56.1.58422: Flags [P.], seq
867930085:867930213, ack 1185651439, win 501, length 128
18:51:41.388447 IP 192.168.56.101.22 > 192.168.56.1.58422: Flags [P.], seq 128:192, ack
1, win 501, length 64
18:51:41.388698 IP 192.168.56.1.58422 > 192.168.56.101.22: Flags [.], ack 192, win 1021,
length 0
18:51:41.388854 IP 192.168.56.101.22 > 192.168.56.1.58422: Flags [P.], seq 192:272, ack
1, win 501, length 80
18:51:41.389135 IP 192.168.56.101.22 > 192.168.56.1.58422: Flags [P.], seq 272:352, ack
1, win 501, length 80
18:51:41.389262 IP 192.168.56.1.58422 > 192.168.56.101.22: Flags [.], ack 352, win 1026,
length 0
18:51:41.389374 IP 192.168.56.101.22 > 192.168.56.1.58422: Flags [P.], seq 352:432, ack
1, win 501, length 80
18:51:41.389588 IP 192.168.56.101.22 > 192.168.56.1.58422: Flags [P.], seq 432:496, ack
1, win 501, length 64
18:51:41.389703 IP 192.168.56.1.58422 > 192.168.56.101.22: Flags [.], ack 496, win 1026,
length 0
18:51:41.480228 IP 192.168.56.101.22 > 192.168.56.1.58422: Flags [P.], seq 496:688, ack
1, win 501, length 192
18:51:41.480415 IP 192.168.56.101.22 > 192.168.56.1.58422: Flags [P.], seq 688:752, ack
1, win 501, length 64
18:51:41.480466 IP 192.168.56.101.22 > 192.168.56.1.58422: Flags [P.], seq 752:912, ack
1, win 501, length 160
18:51:41.480529 IP 192.168.56.1.58422 > 192.168.56.101.22: Flags [.], ack 752, win 1025,
length 0
18:51:41.480607 IP 192.168.56.101.22 > 192.168.56.1.58422: Flags [P.], seq 912:976, ack
1, win 501, length 64
18:51:41.480781 IP 192.168.56.1.58422 > 192.168.56.101.22: Flags [.], ack 976, win 1024,
length 0
18:51:41.480898 IP 192.168.56.101.22 > 192.168.56.1.58422: Flags [P.], seq 976:1136, ack
1, win 501, length 160
18:51:41.481116 IP 192.168.56.101.22 > 192.168.56.1.58422: Flags [P.], seq 1136:1200, ack
1, win 501, length 64
18:51:41.481235 IP 192.168.56.1.58422 > 192.168.56.101.22: Flags [.], ack 1200, win 1023,
length 0
18:51:41.481375 IP 192.168.56.101.22 > 192.168.56.1.58422: Flags [P.], seq 1200:1360,
ack 1, win 501, length 160
18:51:41.481570 IP 192.168.56.101.22 > 192.168.56.1.58422: Flags [P.], seq 1360:1424,
ack 1, win 501, length 64
20 packets captured
68 packets received by filter
0 packets dropped by kernel
```


A4: tcpdump auf dem MQTT-Port (mqtt-vm-a, Interface: enp0s8, Filter: port 1883)

```
usera@mqtt-vm-a:~$ sudo tcpdump -i enp0s8 -n -c 20 port 1883
[sudo] password for usera:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp0s8, link-type EN10MB (Ethernet), snapshot length 262144 bytes
18:53:07.316862 IP 192.168.56.102.46974 > 192.168.56.101.1883: Flags [P.], seq
214377213:214377309, ack 145860215, win 502, options [nop,nop,TS val 644244866 ecr
350208098], length 96
18:53:07.317034 IP 192.168.56.101.1883 > 192.168.56.102.46974: Flags [P.], seq 1:5, ack
96, win 509, options [nop,nop,TS val 350253888 ecr 644244866], length 4
18:53:07.317358 IP 192.168.56.102.46974 > 192.168.56.101.1883: Flags [.], ack 5, win 502,
options [nop,nop,TS val 644244867 ecr 350253888], length 0
18:53:07.321145 IP 192.168.56.101.1883 > 192.168.56.102.46974: Flags [P.], seq 5:101,
ack 96, win 509, options [nop,nop,TS val 350253892 ecr 644244867], length 96
18:53:07.321543 IP 192.168.56.102.46974 > 192.168.56.101.1883: Flags [.], ack 101, win
502, options [nop,nop,TS val 644244871 ecr 350253892], length 0
18:53:07.323213 IP 192.168.56.102.46974 > 192.168.56.101.1883: Flags [P.], seq 96:100,
ack 101, win 502, options [nop,nop,TS val 644244873 ecr 350253892], length 4
18:53:07.364090 IP 192.168.56.101.1883 > 192.168.56.102.46974: Flags [.], ack 100, win
509, options [nop,nop,TS val 350253935 ecr 644244873], length 0
18:53:07.364467 IP 192.168.56.102.46974 > 192.168.56.101.1883: Flags [P.], seq 100:196,
ack 101, win 502, options [nop,nop,TS val 644244914 ecr 350253935], length 96
18:53:07.364495 IP 192.168.56.101.1883 > 192.168.56.102.46974: Flags [.], ack 196, win
509, options [nop,nop,TS val 350253935 ecr 644244914], length 0
18:53:07.364737 IP 192.168.56.101.1883 > 192.168.56.102.46974: Flags [P.], seq 101:105,
ack 196, win 509, options [nop,nop,TS val 350253935 ecr 644244914], length 4
18:53:07.406395 IP 192.168.56.102.46974 > 192.168.56.101.1883: Flags [.], ack 105, win
502, options [nop,nop,TS val 644244956 ecr 350253935], length 0
18:53:08.279202 IP 192.168.56.102.46974 > 192.168.56.101.1883: Flags [P.], seq 196:300,
ack 105, win 502, options [nop,nop,TS val 644245829 ecr 350253935], length 104
18:53:08.279413 IP 192.168.56.101.1883 > 192.168.56.102.46974: Flags [P.], seq 105:109,
ack 300, win 509, options [nop,nop,TS val 350254850 ecr 644245829], length 4
18:53:08.279833 IP 192.168.56.102.46974 > 192.168.56.101.1883: Flags [.], ack 109, win
502, options [nop,nop,TS val 644245829 ecr 350254850], length 0
18:53:08.282054 IP 192.168.56.101.1883 > 192.168.56.102.46974: Flags [P.], seq 109:193,
ack 300, win 509, options [nop,nop,TS val 350254853 ecr 644245829], length 84
18:53:08.282381 IP 192.168.56.102.46974 > 192.168.56.101.1883: Flags [.], ack 193, win
502, options [nop,nop,TS val 644245832 ecr 350254853], length 0
18:53:08.283808 IP 192.168.56.102.46974 > 192.168.56.101.1883: Flags [P.], seq 300:304,
ack 193, win 502, options [nop,nop,TS val 644245833 ecr 350254853], length 4
18:53:08.324214 IP 192.168.56.101.1883 > 192.168.56.102.46974: Flags [.], ack 304, win
509, options [nop,nop,TS val 350254895 ecr 644245833], length 0
18:53:14.890604 IP 192.168.56.102.46970 > 192.168.56.101.1883: Flags [P.], seq
2327694560:2327694562, ack 824296485, win 501, options [nop,nop,TS val 644252440 ecr
350199996], length 2
18:53:14.890735 IP 192.168.56.101.1883 > 192.168.56.102.46970: Flags [P.], seq 1:3, ack
2, win 506, options [nop,nop,TS val 350261461 ecr 644252440], length 2
20 packets captured
21 packets received by filter
0 packets dropped by kernel
```


A6: TCP-Client (nc) (mqtt-vm-b)

```
userb@mqtt-vm-b:~$ printf '%s\n' 'TESTNACHRICHT' | nc 10.0.0.1 12345  
(UNKNOWN) [10.0.0.1] 12345 (?): Connection refused  
userb@mqtt-vm-b:~$
```

Performance und Latenz

A1: Start des MQTT-Tunnel-Servers - Secure- und Baseline Mode Mode (mqtt-vm-a)

```
sudo ./mqtt_tunnel --mode server --broker 192.168.56.101 --client-id tunnel-server-a --  
command-channel tunnelA -v
```

B1.1: Start des MQTT-Tunnel-Servers - Baseline Mode (mqtt-vm-b)

```
sudo ./mqtt_tunnel --mode client --broker 192.168.56.101 --client-id tunnel-client-b --insecure  
--command-channel tunnelA -v
```

B1.2: Start des MQTT-Tunnel-Servers - Secure Mode (mqtt-vm-b)

```
sudo ./mqtt_tunnel --mode client --broker 192.168.56.101 --client-id tunnel-client-b --  
command-channel tunnelA -v
```

Zusatz: identity.pub im aktuellen Arbeitsverzeichnis vorhanden

A2.1: iperf3 Server (mqtt-vm-a)

```
sudo ./mqtt_tunnel --mode server --broker 192.168.56.101 --client-id tunnel-server-a --  
command-channel tunnelA -v  
Server listening on 5201
```

A2.2: Crypto-Nachweis Empfang (mqtt-vm-a)

Baseline

```
nc -l -p 12345
```

Ausgabe

```
TESTNACHRICHT
```

Secure Ausgabe (keine Ausgabe)

tcpdump Capture (physisches Interface)(mqtt-vm-a)

Baseline:

```
sudo tcpdump -i enp0s8 -s0 -w mqtt_baseline.pcap port 1883
```

Secure:

```
sudo tcpdump -i enp0s8 -s0 -w mqtt_secure.pcap port 1883
```

B2.1: TCP-Durchsatzmessung (Baseline, Client → Server) (mqtt-vm-b)

```
iperf3 -c 10.0.0.1 -t 30
```

Ausgabe

```
[ 5] 0.00-30.00 sec 31.1 MBytes 8.70 Mbits/sec 460 sender  
[ 5] 0.00-31.19 sec 29.8 MBytes 8.00 Mbits/sec receiver
```

B2.2: TCP Reverse (Baseline) (mqtt-vm-b)

```
iperf3 -c 10.0.0.1 -R -t 30 -w 64K
```

Ausgabe

```
[ 5] 0.00-30.19 sec 9.23 MBytes 2.56 Mbits/sec sender  
[ 5] 0.00-30.00 sec 9.17 MBytes 2.56 Mbits/sec receiver
```

B2.3: UDP-Messung 10 Mbit/s (Baseline) (mqtt-vm-b)

```
iperf3 -c 10.0.0.1 -u -b 10M -t 30
```

Ausgabe

```
[ 5] receiver 5.90 Mbits/sec 9829/25769 (38%)
```

B2.4: TCP-Durchsatzmessung (Secure, Client → Server) (mqtt-vm-b)

```
iperf3 -c 10.0.0.1 -t 30
```

Ausgabe

```
[ 5] 0.00-30.00 sec 24.1 MBytes 6.75 Mbits/sec 708 sender  
[ 5] 0.00-30.83 sec 21.7 MBytes 5.91 Mbits/sec receiver
```

B2.5: UDP-Messung 10 Mbit/s (Secure) (mqtt-vm-b)

```
iperf3 -c 10.0.0.1 -u -b 10M -t 30
```

Ausgabe

```
[ 5] receiver 7.38 Mbits/sec 6062/25898 (23%)
```

B2.6: CPU-Auslastung während Secure TCP Forward (mqtt-vm-b)

```
pidstat -p $(pgrep -n mqtt_tunnel) 1
```

Ausgabe

```
%CPU 25-35 %
```

B2.7: MSS-Gegenprobe (Secure TCP Forward) (mqtt-vm-b)

```
iperf3 -c 10.0.0.1 -t 30 -w 64K --set-mss 1200
```

Ausgabe

```
2.14 Mbits/sec receiver
```

```
iperf3 -c 10.0.0.1 -t 30 -w 64K --set-mss 1400
```

Ausgabe

```
5.27 Mbits/sec receiver
```

B2.8: Crypto-Nachweis (strings-Analyse) (mqtt-vm-b)

Baseline

```
printf 'TESTNACHRICHT\n' | nc 10.0.0.1 12345
```

Secure

```
printf 'TESTNACHRICHT\n' | nc 10.0.0.1 12345
```